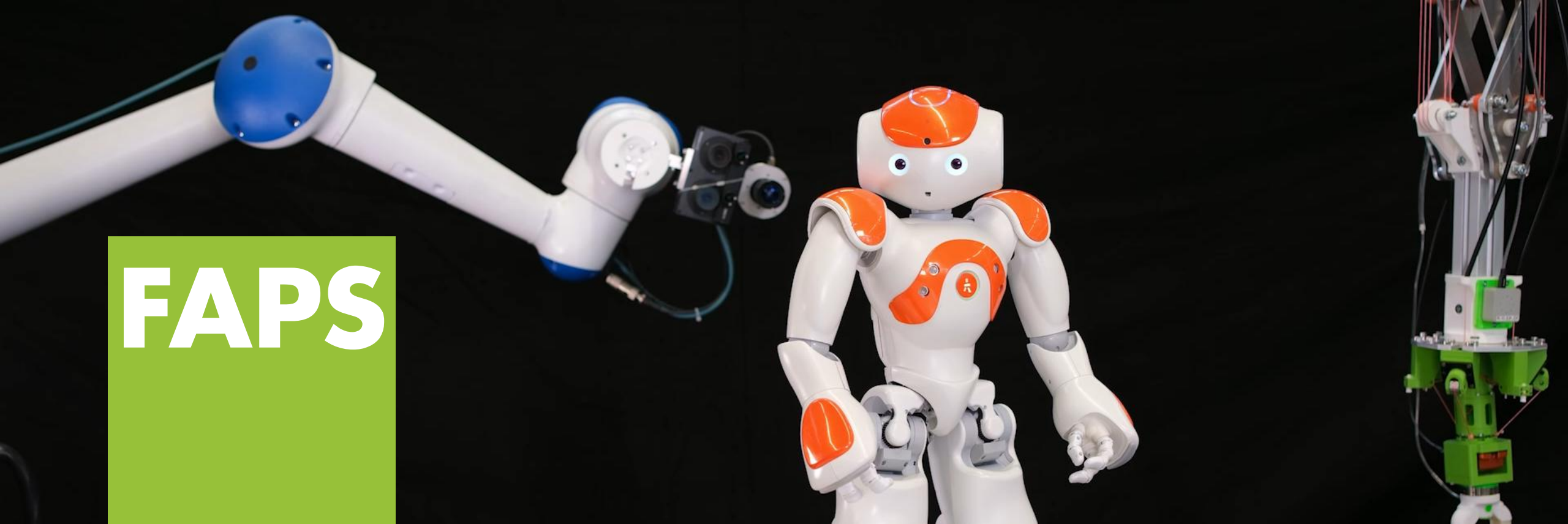




Prof. Dr.-Ing. Jörg Franke

**Lehrstuhl für Fertigungsautomatisierung
und Produktionssystematik**

Friedrich-Alexander-Universität Erlangen-Nürnberg



**Friedrich-Alexander-Universität
Technische Fakultät**

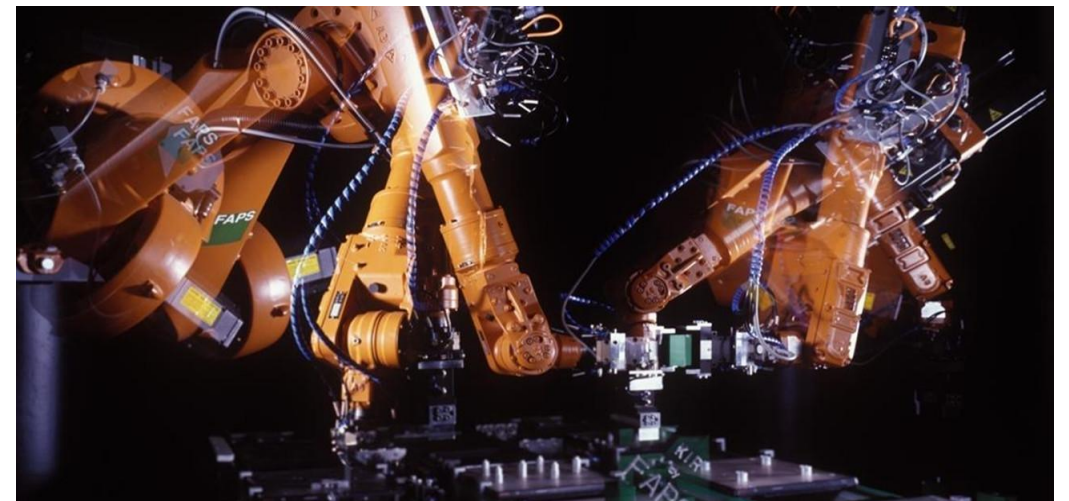
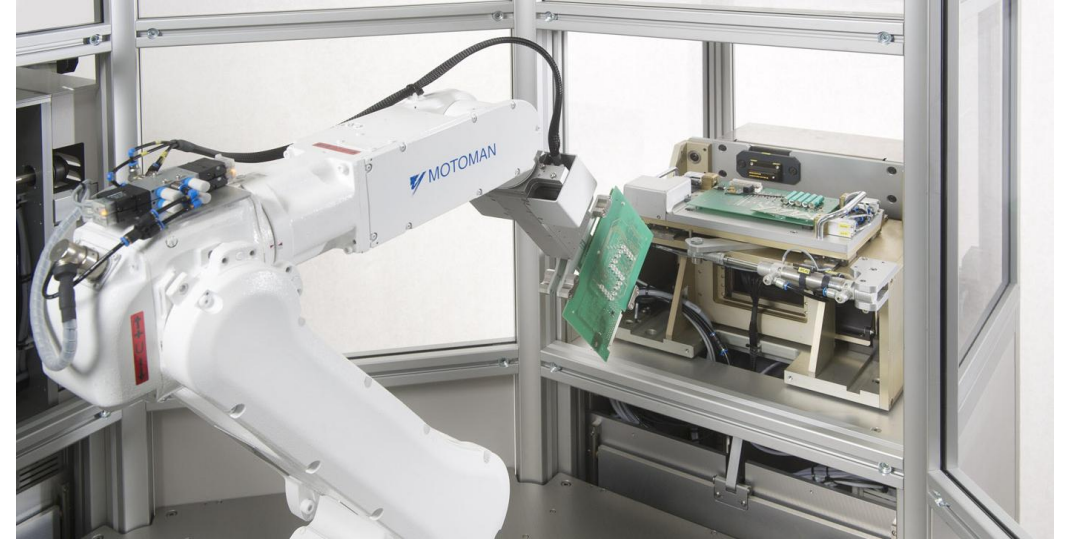
Grundlagen der Robotik (GdR)

VE06 – Grundlagen des Robot Operating Systems (ROS)

Im Fach Grundlagen der Robotik werden die fachspezifischen Grundlagen zur Konzeption, Implementierung und Realisierung von Robotersystemen vermittelt.

VE	Thema
----	-------

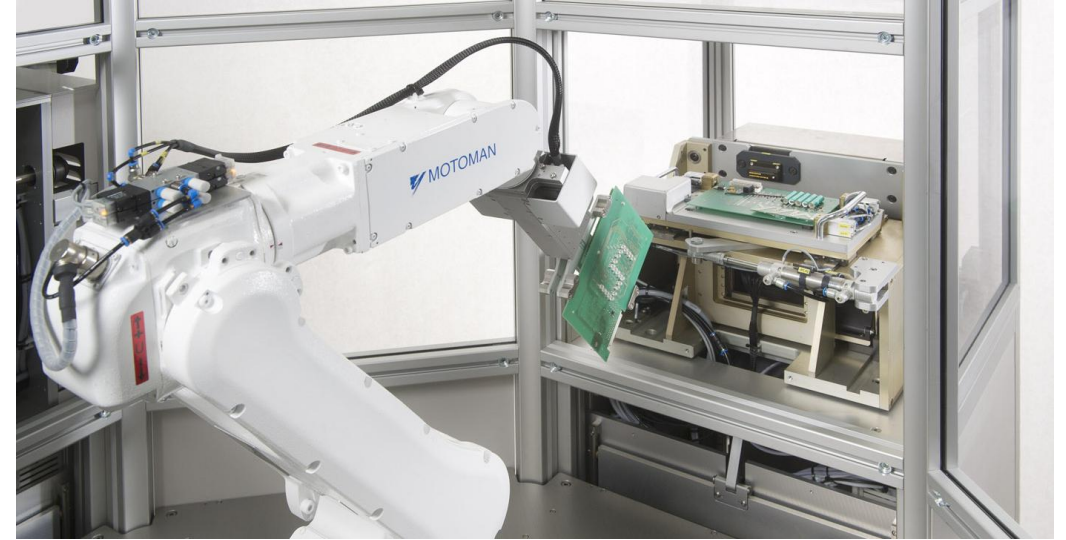
- | | |
|----|--|
| 1 | Einführung und Grundlagen der Industrierobotik & Neuartige Konzepte und Kinematiken in der Robotik |
| 2 | Steuerung - Sensorik – Aktorik |
| 3 | Roboterkinematik |
| 4 | Roboterdynamik |
| 5 | Bahnplanung und Regelung |
| 6 | Grundlagen des Robot Operating System (ROS) |
| 7 | ROS - Erweiterte Konzepte und Tools |
| 8 | Simulation und Planung |
| 9 | Rechnersehen |
| 10 | Maschinelles Lernen |
| 11 | Seilrobotik |
| 12 | Service-, Pflege- und Medizinrobotik |



Die zugehörige Übung setzt sich aus Tafel- und Hausübungen zusammen und fokussiert unter anderem die praktische Arbeit mit dem Robot Operating System.

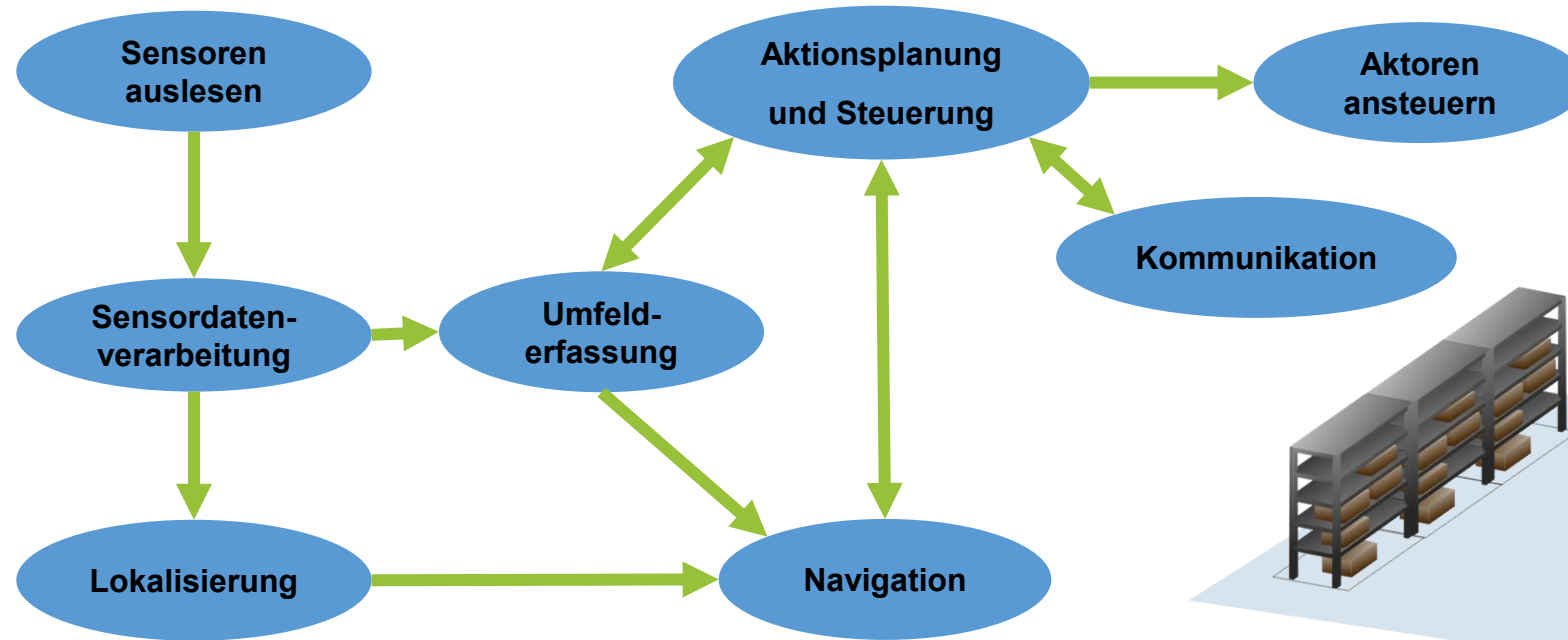
UE	Thema
----	-------

- | | |
|----|---|
| 1 | Hallenführung in Kleingruppen |
| 2 | Steuerung - Sensorik - Aktorik |
| 3 | Roboterkinematik |
| 4 | Roboterdynamik |
| 5 | Bahnplanung und Regelung |
| 6 | Grundlagen des Robot Operating System (ROS) |
| 7 | ROS - Erweiterte Konzepte und Tools |
| 8 | Simulation und Planung |
| 9 | Computer Vision |
| 10 | Machine Learning |



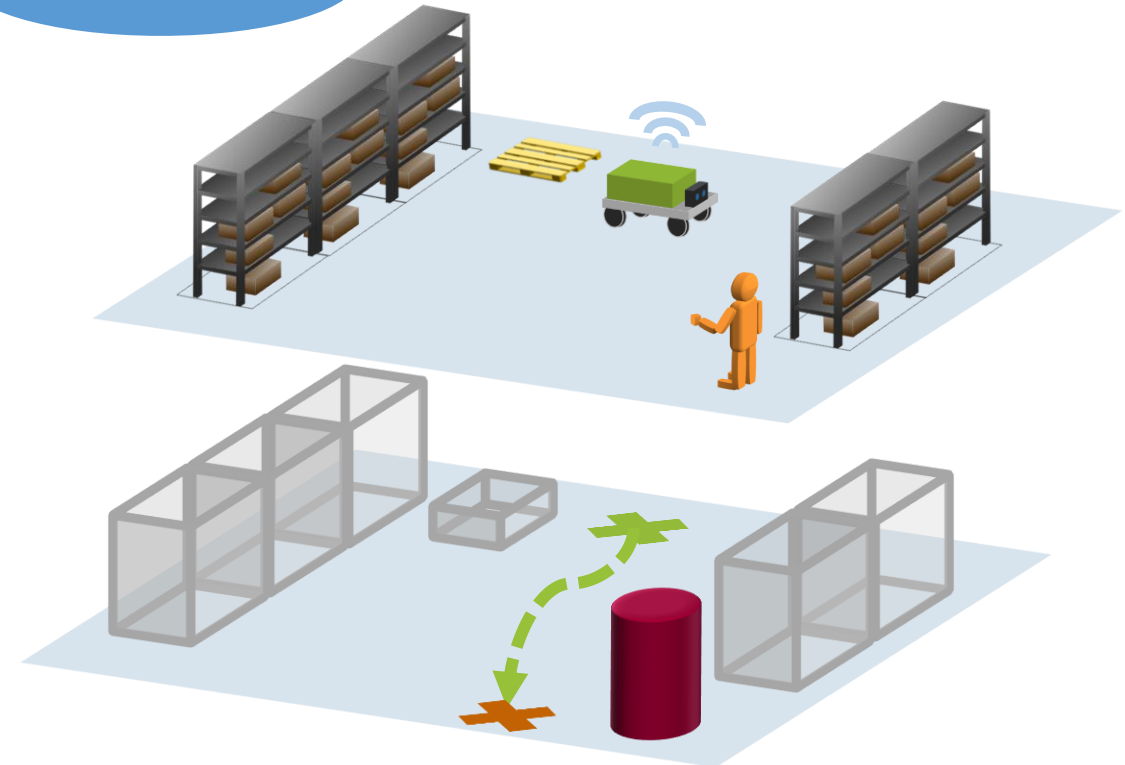
Im Rahmen der Tafelübungen und der zugehörigen Hausübungen werden einfache Programmierbeispiele mit Python sowie die praktische Anwendung des Robot Operating System (ROS) behandelt.

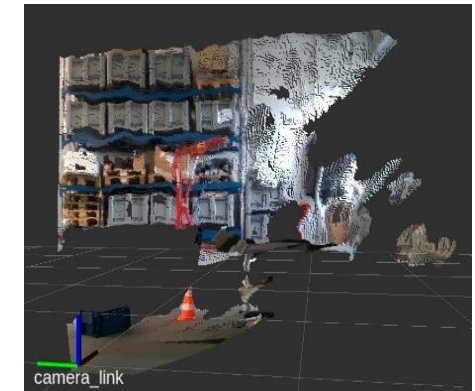
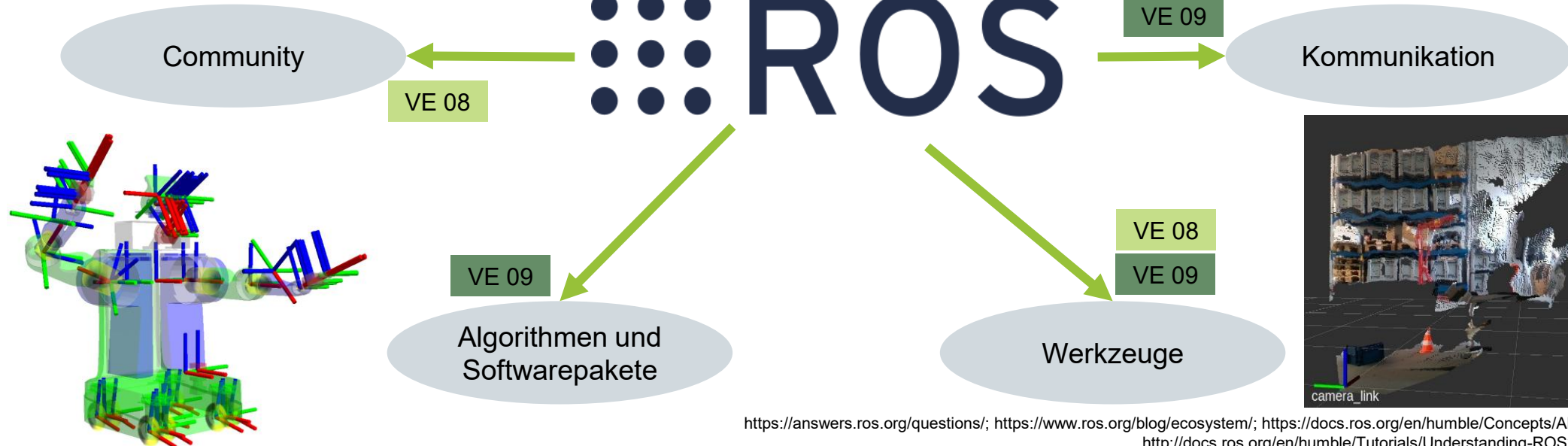
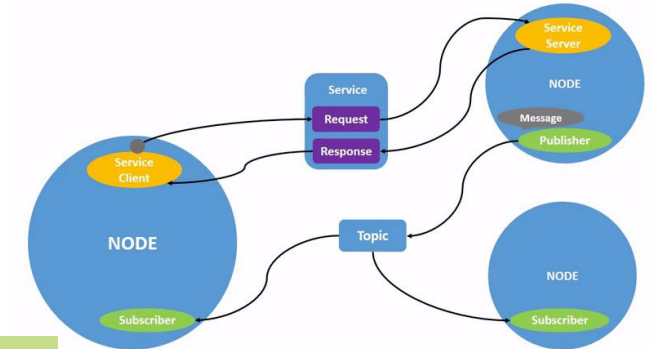
Ein autonomer Roboter muss zur sicheren und zuverlässigen Erledigung seiner Aufgaben viele unterschiedliche Fähigkeiten besitzen.



Die Software soll

- robust und zuverlässig,
- mit unterschiedlichen Systemen kompatibel,
- wiederverwendbar
- und verständlich sein.





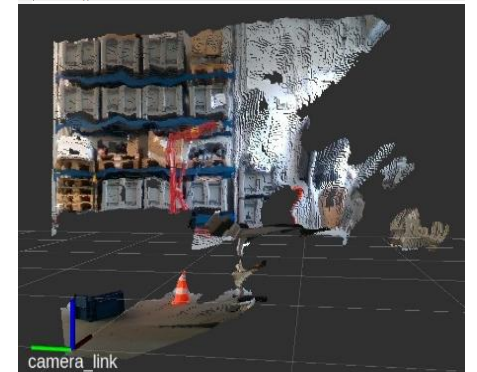
<https://answers.ros.org/questions/>; <https://www.ros.org/blog/ecosystem/>; <https://docs.ros.org/en/humble/Concepts/About-Tf2.html>
<http://docs.ros.org/en/humble/Tutorials/Understanding-ROS2-Nodes.html>

Die Vorlesungseinheit „Grundlagen des Robot Operating System (ROS)“ soll eine Übersicht zum ROS geben sowie die Grundlagen zur Arbeit mit dem ROS vermitteln.

Das **Robot Operating System** ist ein **Meta-Betriebssystem** zur Entwicklung von **Software** für Roboter und zum **Betrieb** von **Robotern**.



```
1 import rclpy
2 from rclpy.node import Node
3 from std_msgs.msg import String
4
5 class GdrPublisher(Node):
6
7     def __init__(self):
8         super().__init__('gdr_publisher')
9         self.publisher_ = self.create_publisher(String, 'topic', 10)
10        timer_period = 0.5 # seconds
11        self.timer = self.create_timer(timer_period, self.timer_callback)
12        self.i = 0
13
14    def timer_callback(self):
15        msg = String()
16        msg.data = 'Hello World: %d' % self.i
17        self.publisher_.publish(msg)
18        self.get_logger().info('Publishing: "%s"' % msg.data)
19        self.i += 1
20
21 def main(args=None):
22     rclpy.init(args=args)
23
24     gdr_publisher = GdrPublisher()
25
26     rclpy.spin(gdr_publisher)
27
28     # Destroy the node explicitly
29     # (optional - otherwise it will be done automatically
30     # when the garbage collector destroys the node object)
31     gdr_publisher.destroy_node()
32     rclpy.shutdown()
33
34 if __name__ == '__main__':
35     main()
```



Nach dem Besuch dieser Vorlesungseinheit sollten Sie in der Lage sein:

- die wesentlichen **Aspekte** und die **Philosophie** des ROS zu beschreiben,
- die **grundlegende Strukturierung** des ROS einzuordnen,
- die **Software** für ein **einfaches Robotersystem** mithilfe des ROS zu konzeptionieren,
- die **elementaren Kommunikationsmechanismen** des ROS zu erläutern,
- einfache **Programme** zur **Steuerung** und **Kommunikation** zu entwickeln sowie
- grundlegende **Werkzeuge** zur **Analyse** des **ROS** nutzen zu können.

Umfangreiche Zusatzinformationen zu den Inhalten der Vorlesung, Übung und Hausübung sind in der ROS-Dokumentation zu finden und können zum Selbststudium genutzt werden.

- Vorlesung: Theorie und Hintergründe (Prüfungsrelevant)
- Übung: Praktische Anwendung (Prüfungsrelevant)
- Hausübungen: Selbstständige Vertiefung des Gelernten (Freiwillig)

- Weitere Informationen:
 - <https://www.ros.org/>
 - <http://docs.ros.org/en/humble/index.html>
 - <https://design.ros2.org/>
 - Macenski, S., Foote, T., Gerkey, B., Lalancette, C., & Woodall, W. (2022). Robot Operating System 2: Design, architecture, and uses in the wild. Science Robotics, 7 (66)

Das Robot Operating System ist ein Meta-Betriebssystem zur Entwicklung von Software für Roboter und zum Betrieb von Robotern.

- **Überblick über das ROS 2**

- Packages

- Nodes

- Kommunikation

- Topics
- Messages
- Services

- Parameter

- ROS Community

- Anwendungsbeispiele



<https://github.com/ros-infrastructure/artwork/blob/master/distributions/humble/HumbleHawksbill.png>

Das ROS ist ein Meta-Betriebssystem für die Robotik, welches oft in Kombination mit Ubuntu eingesetzt wird.



Ubuntu

Quelle: <https://design.ubuntu.com/downloads/>

- Viele Softwarepakete nur unter Ubuntu getestet / verfügbar
- Kostenlos verfügbar
- Effiziente Entwicklungsumgebung
- Große Community

Weitere Betriebssysteme:

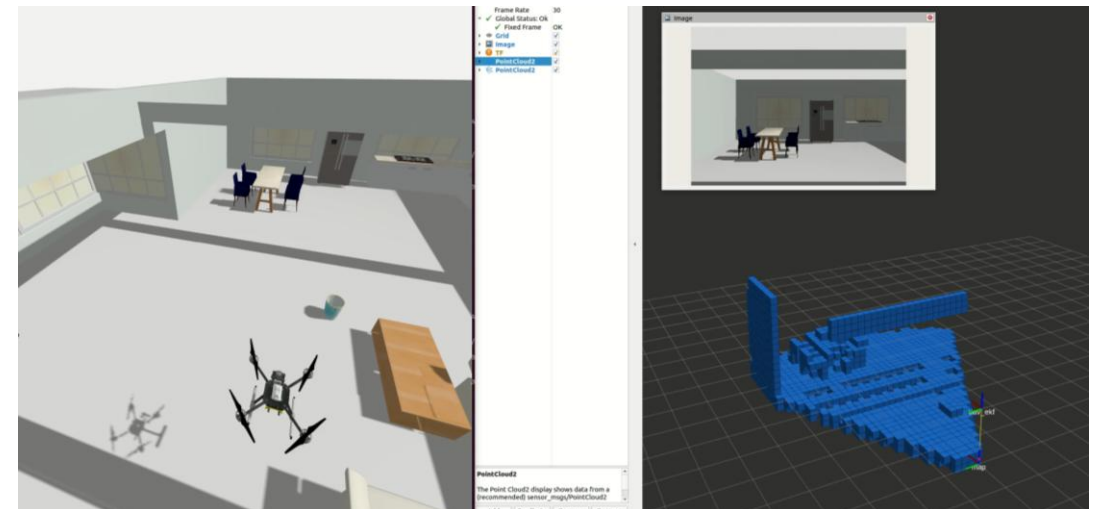
- Windows
- macOS
- RHEL
- Debian

<http://docs.ros.org/en/humble/Installation.html>

Das ROS als Meta-Betriebssystem erweitert die Fähigkeiten des zu Grunde liegenden Betriebssystems.

Meta-Betriebssystem:

- **Gerätetreiber:**
ROS bietet Gerätetreiber für Sensoren, Aktoren und weitere Robotik-Hardware.
- **Hardwareabstraktion:**
ROS bietet Schnittstellen zu unterschiedlichster Hardware und vereinheitlicht die Softwareinteraktion damit.
- **Paketmanagement:**
ROS unterstützt das Installieren, Aktualisieren und Deinstallieren von ROS-Softwarepaketen.
- **Interprozesskommunikation:**
ROS ermöglicht die Kommunikation zwischen unterschiedlichen Programmen.
- **Werkzeuge und Bibliotheken:**
ROS stellt unterschiedliche Werkzeuge und Programmbibliotheken zur Verfügung.



<http://wiki.ros.org/ROS/Introduction>

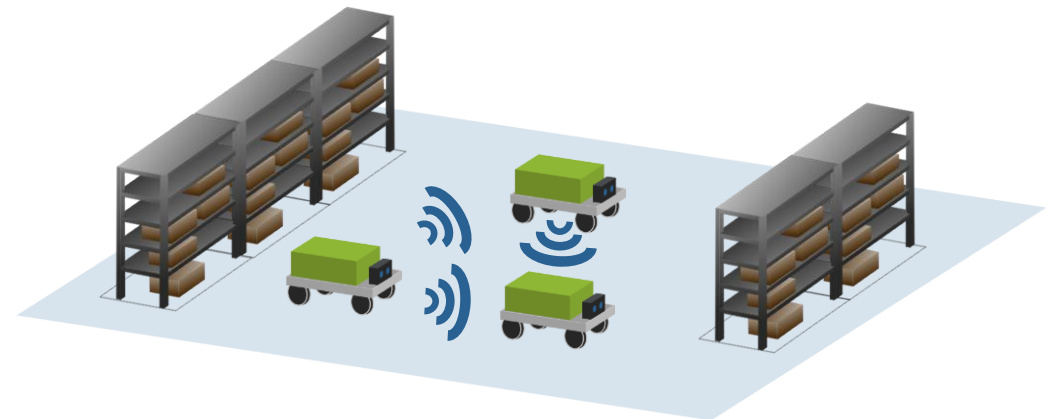
Das ROS ist speziell auf die Bedürfnisse der Robotik abgestimmt.

Philosophie:

- **Verteiltes System:**
Programme können auf unterschiedlichen Rechnern ausgeführt werden und über ein Netzwerk miteinander kommunizieren.
- **Peer-to-Peer:**
Programme kommunizieren auf direktem Weg miteinander.
- **Multilingual:**
ROS Programme können in verschiedenen Programmiersprachen verfasst werden (**C++**, **Python**, Java, Matlab, C#, Rust, ...).
- **Thin:**
Software-Bibliotheken sollen eigenständig erstellt werden und dann in ROS Programme eingebunden werden (Beispiel: OpenCV).
- **Kostenfrei und Open-Source**

Neue Aspekte des ROS2:

- Roboterteams
- Embedded-Plattformen
- Echtzeitsysteme
- Nicht ideale Netzwerke
- Produktionsumgebungen
- Muster für die Erstellung und Strukturierung von Software



https://design.ros2.org/articles/why_ros2.html
<https://docs.ros.org/en/humble/Concepts/About-ROS-2-Client-Libraries.html>

Das Robot Operating System wird schon seit vielen Jahren entwickelt und genutzt.

ROS Noetic Ninjemys (Recommended)	May 23rd, 2020			May, 2025 (Focal EOL)
ROS Melodic Morenia	May 23rd, 2018			May, 2023 (Bionic EOL)
ROS Lunar Loggerhead	May 23rd, 2017			May, 2019
ROS Kinetic Kame	May 23rd, 2016			April, 2021 (Xenial EOL)
ROS Jade Turtle	May 23rd, 2015			May, 2017
ROS Indigo Igloo	July 22nd, 2014			April, 2019 (Trusty EOL)
ROS Hydro Medusa	September 4th, 2013			May, 2015

ROS Groovy Galapagos	December 31, 2012			July, 2014
ROS Fuerte Turtle	April 23, 2012			--
ROS Electric Emys	August 30, 2011			--
ROS Diamondback	March 2, 2011			--
ROS C Turtle	August 2, 2010			--
ROS Box Turtle	March 2, 2010			--
		Box Turtle		

<http://wiki.ros.org/Distributions>

ROS 2 wird als Nachfolger von ROS1 federführend von der Open Source Robotics Foundation entwickelt und verwaltet.

Distro	Release date	Logo	EOL date
Kilted Kaiju	May 23, 2025		December 2026
Jazzy Jalisco	May 23, 2024		May 2029
Iron Irwini	May 23, 2023		December 4, 2024
Humble Hawksbill	May 23, 2022		May 2027
Galactic Geochelone	May 23, 2021		December 9, 2022
Foxy Fitzroy	June 5, 2020		June 20, 2023
Eloquent Elusor	November 22, 2019		November 2020
Dashing Diademata	May 31, 2019		May 2021

Crystal Clemmys	December 14th, 2018		December 2019
Bouncy Bolson	July 2nd, 2018		July 2019
Ardent Apalone	December 8th, 2017		December 2018
beta3	September 13th, 2017		December 2017
beta2	July 5th, 2017		September 2017
beta1	December 19th, 2016		Jul 2017
alpha1 - alpha8	August 31th, 2015		December 2016



<https://docs.ros.org/en/rolling/Releases.html>
<https://www.openrobotics.org/>

Das Robot Operating System ist ein Meta-Betriebssystem zur Entwicklung von Software für Roboter und zum Betrieb von Robotern.

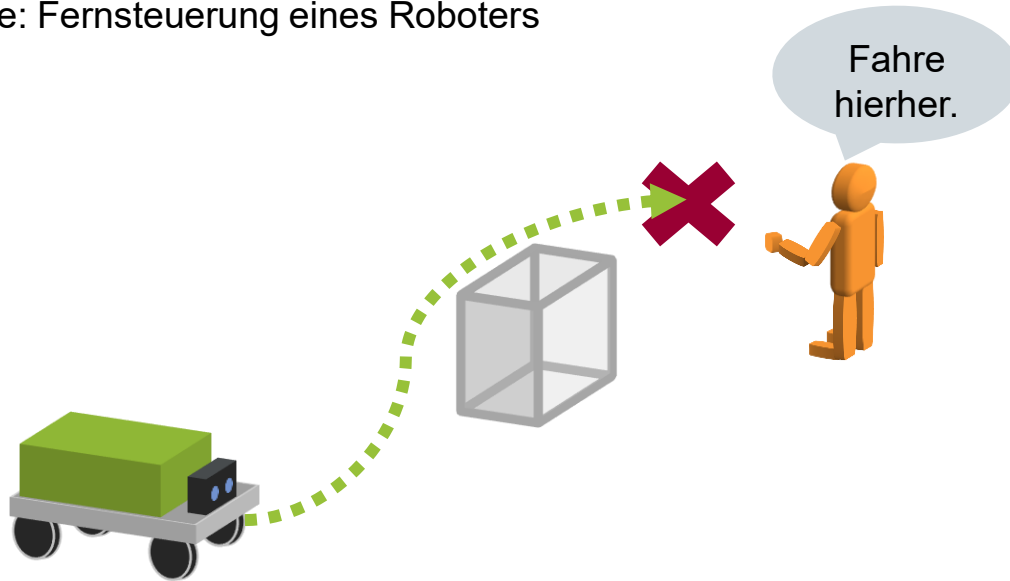
- Überblick über das ROS 2
- **Packages**
- Nodes
- Kommunikation
 - Topics
 - Messages
 - Services
- Parameter
- ROS Community
- Anwendungsbeispiele



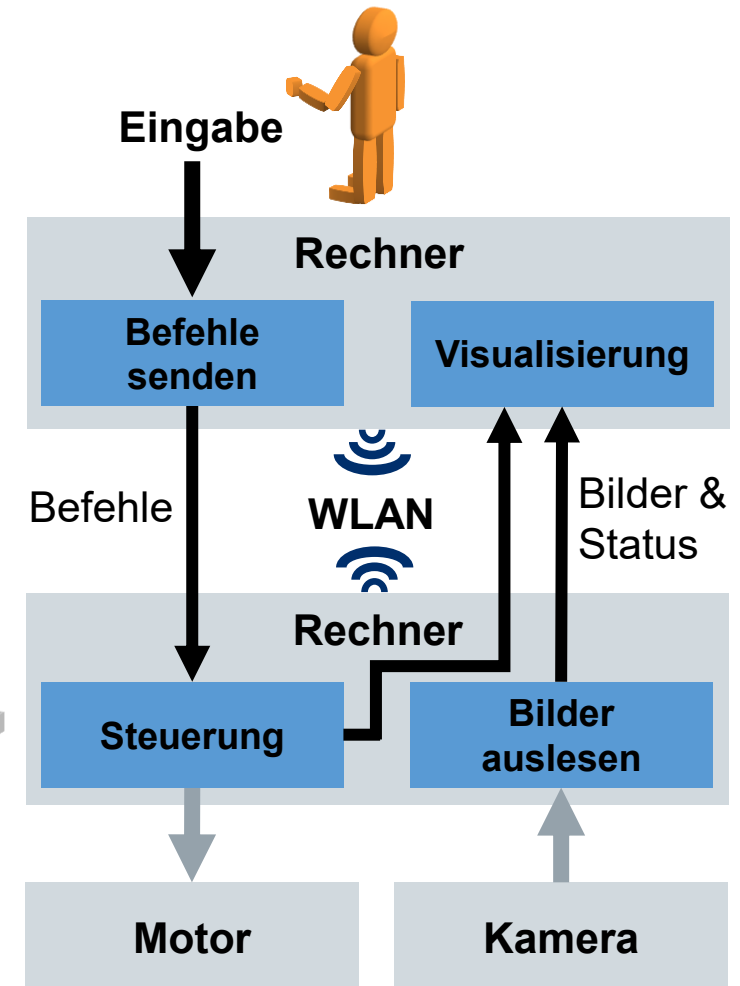
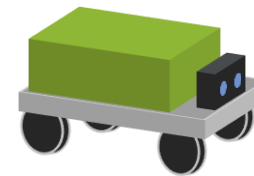
<https://github.com/ros-infrastructure/artwork/blob/master/distributions/humble/HumbleHawksbill.png>

In dieser Vorlesungseinheit werden die grundlegenden Funktionen des ROS anhand eines einfachen Beispiels vorgestellt.

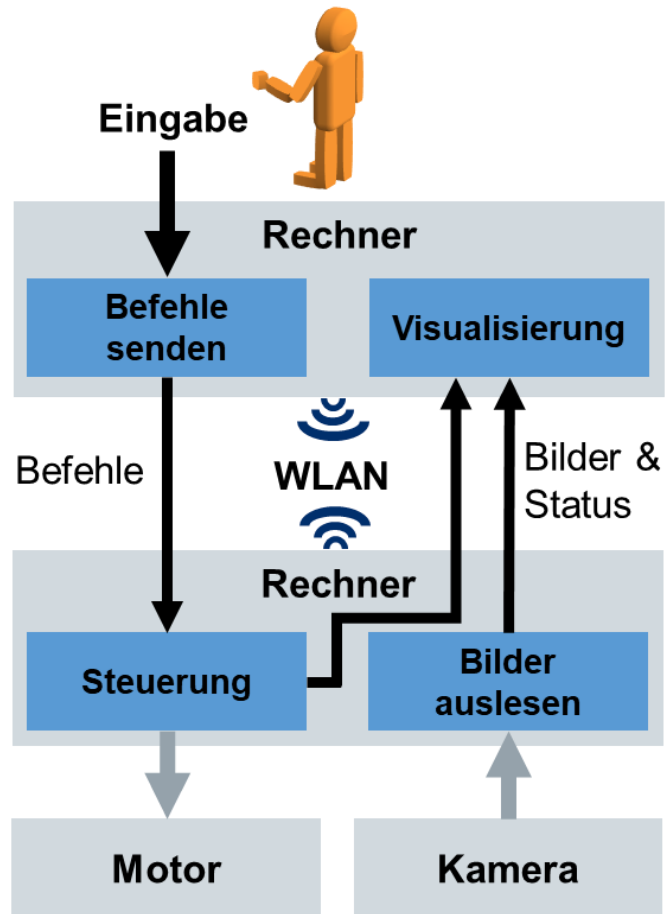
- Aufgabe: Fernsteuerung eines Roboters



ROS



Die Funktionalitäten und Kommunikationsfähigkeiten werden in Packages aufgeteilt.



Befehle senden

- Eingabe mit Tastatur
- Eingabe in Geschwindigkeit umwandeln
- Geschwindigkeit an Roboter via WLAN senden

Visualisierung

- Akkustand, aktuelle Geschwindigkeit und Bild empfangen
- Akkustand und aktuelle Geschwindigkeit als Text anzeigen
- Bild anzeigen

Steuerung

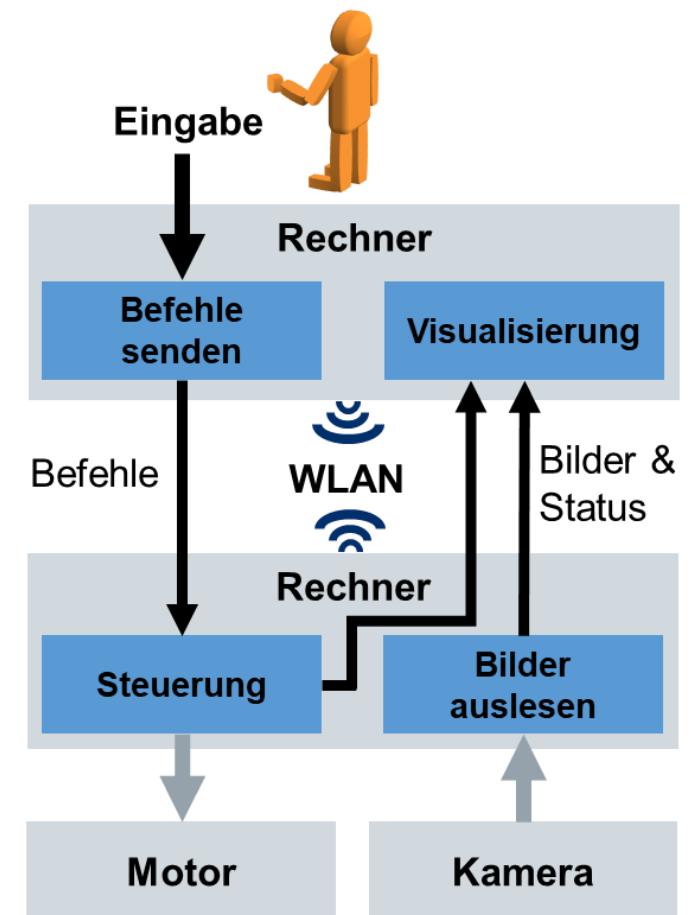
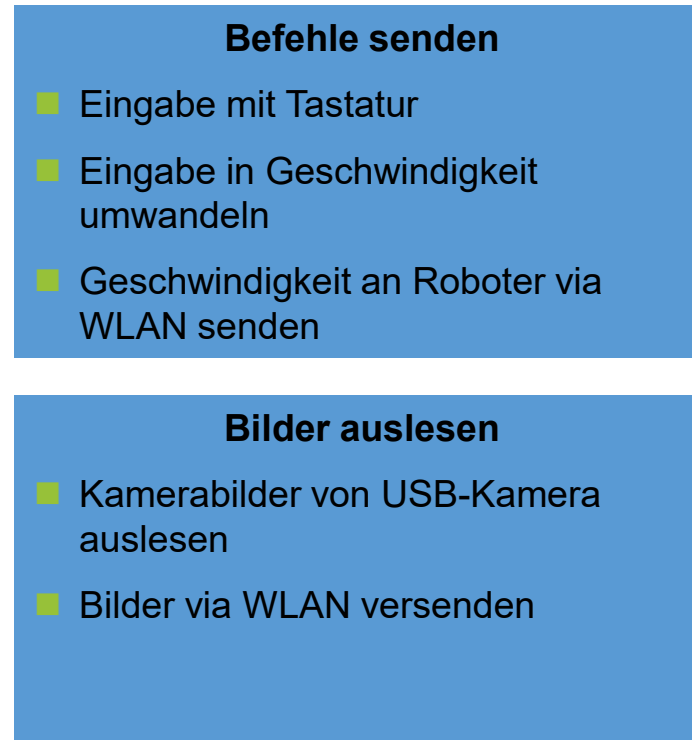
- Befehle empfangen
- Motoren ansteuern
- Akkustand und aktuelle Geschwindigkeit via WLAN versenden

Bilder auslesen

- Kamerabilder von USB-Kamera auslesen
- Bilder via WLAN versenden

Packages strukturieren die Funktionalitäten eines Roboters.

- Packages dienen zum Organisieren und Erstellen von
 - Software
 - Bibliotheken
 - Funktionalitäten
 - Kommunikationsschnittstellen
 - und Konfigurationsdateien
- Ein ROS Package kann in jeder unterstützten Sprache entwickelt und dann sprachunabhängig genutzt werden
- Packages sind wiederverwendbar
- Inhalt:
 - package.xml: Datei mit Metainformationen zum Package
 - setup.py: Instruktionen zum Installieren des Packages
 - setup.cfg: Informationen zu ausführbaren Programme des Packages
 - <package_name>: Ordner mit Code



<https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries/Creating-Your-First-ROS2-Package.html>

Ein Package enthält unter anderem
Programmcode, Anweisung zum Erstellen des Codes und Bibliotheken.

Package teleop

- Selbst entwickeltes Package
- Enthält Software (Python)

VE 08

Package robot_control

- Selbst entwickeltes Package
- Enthält Software (Python)

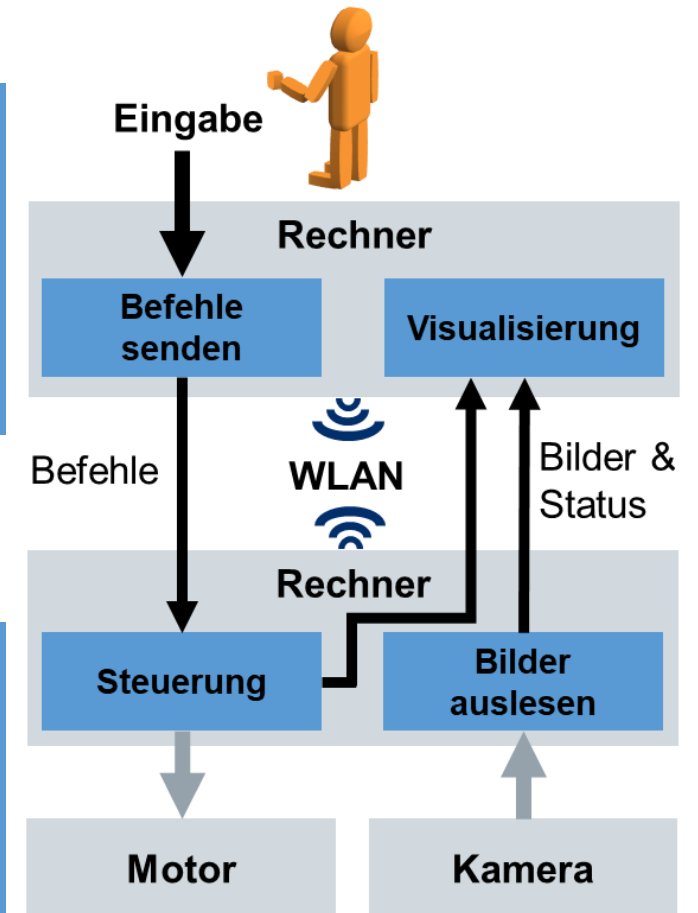
Visualisierung mit dem Tool rviz2

- Nutzung von ROS Tools
- ROS Tools sind auch als Packages strukturiert

VE 09

Package usb_cam

- Nutzung eines Community Packages
- Community Packages können einfach in das eigene System eingebunden werden



<https://index.ros.org/p/rviz2/#humble>
https://github.com/ros-drivers/usb_cam

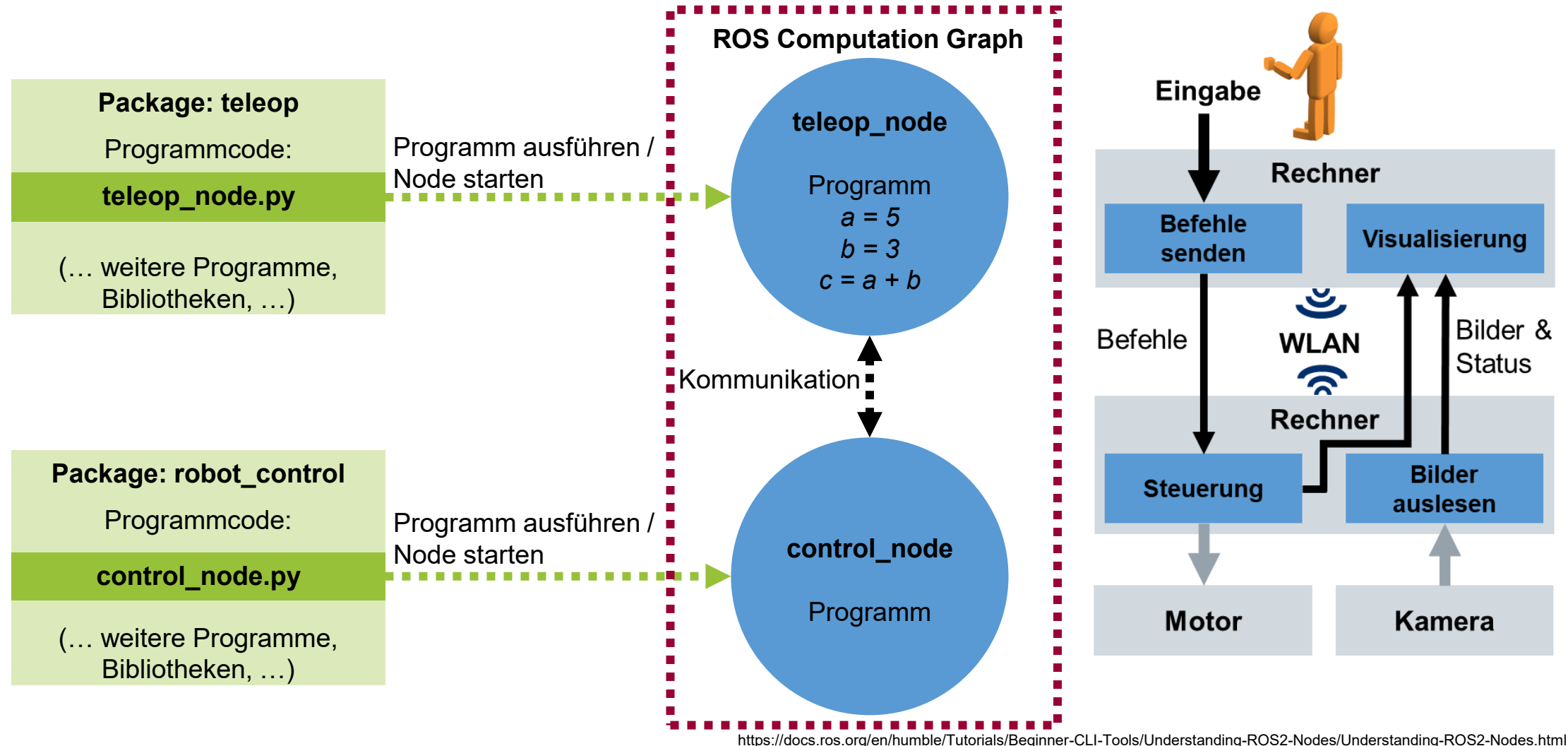
Das Robot Operating System ist ein Meta-Betriebssystem zur Entwicklung von Software für Roboter und zum Betrieb von Robotern.

- Überblick über das ROS 2
- Packages
- **Nodes**
- Kommunikation
 - Topics
 - Messages
 - Services
- Parameter
- ROS Community
- Anwendungsbeispiele



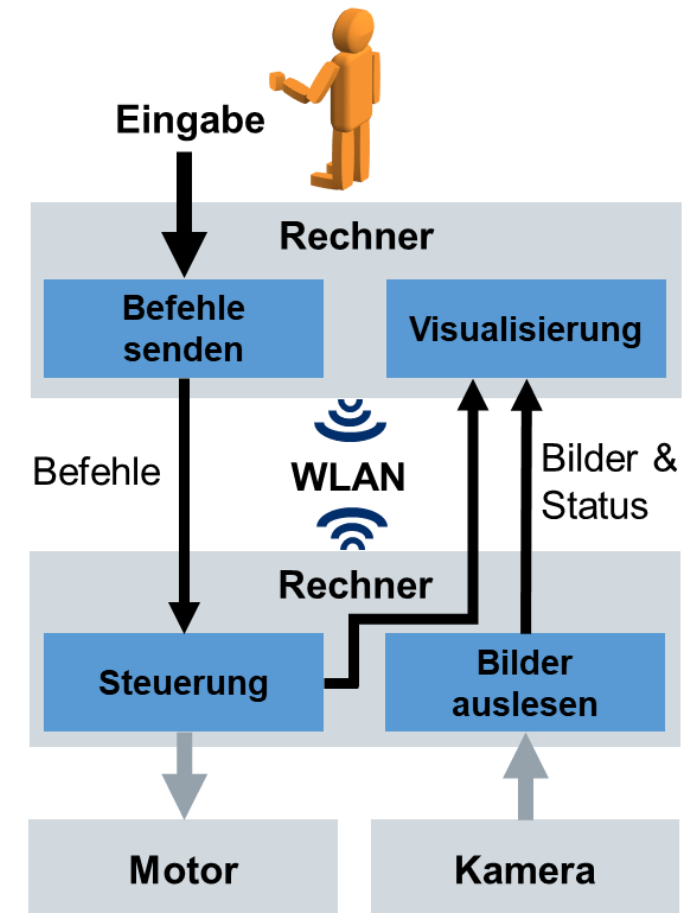
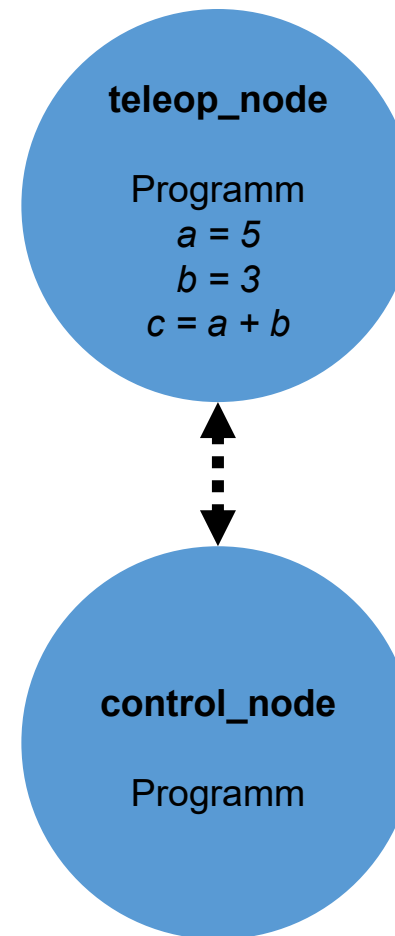
<https://github.com/ros-infrastructure/artwork/blob/master/distributions/humble/HumbleHawksbill.png>

Ein ROS Package kann Programme beinhalten,
die mit Hilfe des ROS ausgeführt werden können.



Ein ROS Node ist ein Prozess, der Berechnungen ausführt.

- Ein Node implementiert eine Funktionalität
- Ein Node kann innerhalb des ROS Systems mit anderen Nodes kommunizieren
- Ein Node kann in jeder von ROS unterstützten Sprache implementiert werden
- Die Implementierung eines Nodes erfolgt mithilfe einer ROS Client Library (rcl), z.B.: **rclcpp**, **rclpy**
- Vorteile des Knotenprinzips:
 - Höhere Ausfallsicherheit
 - Geringere Komplexität der einzelnen Nodes
 - Beliebige Nodes können durch entsprechende Kommunikationsschnittstellen kommunizieren
 - Einfacheres Verständnis des Gesamtsystems



<http://wiki.ros.org/ROS/Concepts>

<http://wiki.ros.org/Nodes>

<https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Nodes/Understanding-ROS2-Nodes.html>

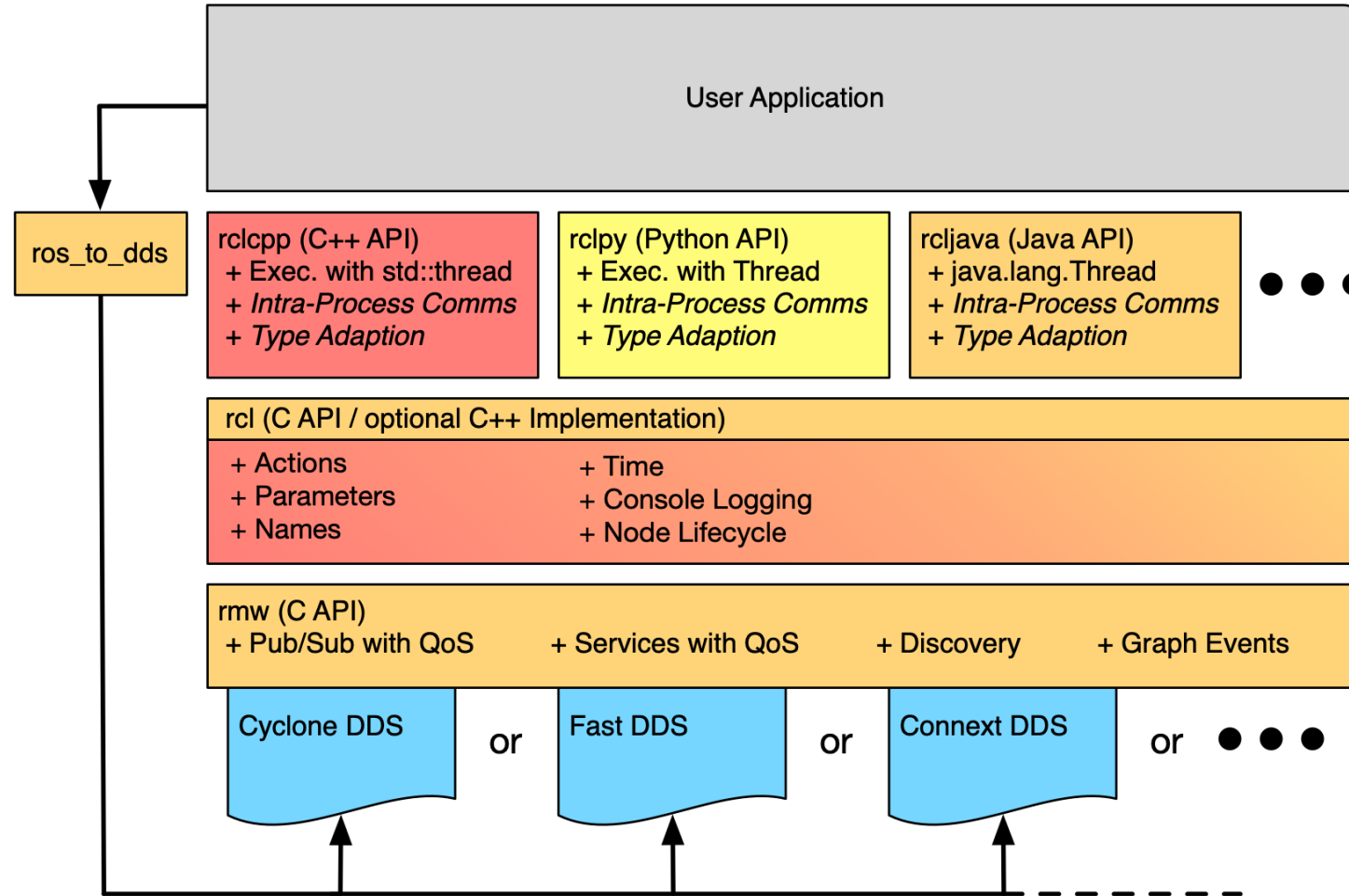
Das Robot Operating System ist ein Meta-Betriebssystem zur Entwicklung von Software für Roboter und zum Betrieb von Robotern.

- Überblick über das ROS 2
- Packages
- Nodes
- **Kommunikation**
 - Topics
 - Messages
 - Services
- Parameter
- ROS Community
- Anwendungsbeispiele



<https://github.com/ros-infrastructure/artwork/blob/master/distributions/humble/HumbleHawksbill.png>

Die Kommunikation des ROS 2 basiert auf dem Data Distribution Service (DDS) Standard.



* *Intra-Process Comms* and *Type Adaption* could be implemented in the client library, but may not currently exist.

Data Distribution Service (DDS):

- Standard der Object Management Group (OMG)
- Middleware basierend auf dem Publish-Subscribe Muster für die Maschine-Maschine-Kommunikation
- Implementierungen unterschiedlicher Anbieter verfügbar, z.B.: eProsima Fast DDS, Eclipse Cyclone DDS, RTI Connex DDS
- Schnittstelle ROS middleware interface (rmw)
- Quality of Service (QoS): Einstellung unterschiedlicher Parameter für die Kommunikation möglich
- Discovery: Die Nodes finden sich selbstständig und bauen Kommunikationsverbindungen auf

https://design.ros2.org/articles/ros_on_dds.html
<https://docs.ros.org/en/humble/Concepts/About-Different-Middleware-Vendors.html>

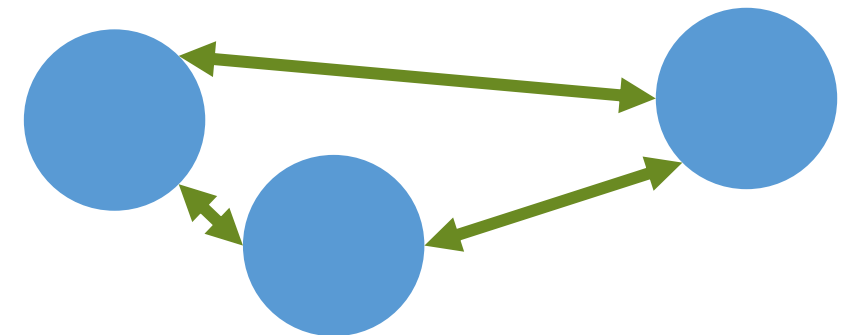
Der Discovery-Prozess ermöglicht Verbindungen und Kommunikation ohne ein zentrales Element.

Quality of Service (QoS):

- History
 - Keep last: Es werden nur N Messages beibehalten.
 - Keep all: Es werden alle Messages beibehalten (abhängig von den Ressourcen der Middleware).
- Reliability
 - Best effort: Es wird versucht, die Messages zuzustellen, ein Verlust ist aber abhängig von der Netzwerkrobustheit möglich.
 - Reliable: Es wird garantiert, dass die Messages zugestellt werden, ggf. mit mehreren Zustellversuchen.
- Durability
 - Transient local: Der Publisher speichert Messages auch für „verspätete“ Subscriptions.
 - Volatile: Es werden keine Messages gespeichert.
- QoS Profile: Voreingestellte Parameter für unterschiedliche Anwendungsszenarien
 - Default QoS settings for publishers and subscriptions (Keep last, Reliable, Volatile)
 - Sensor data (best effort reliability und kleinere queue size)
 - ...

Discovery:

- Sobald ein Node gestartet wird, benachrichtigt er die anderen Nodes im Netzwerk über seine Präsenz, welche über dieselbe ROS domain ID verfügen
- Die anderen Nodes benachrichtigen den neuen Node mit Informationen zu sich selbst, sodass Verbindungen aufgebaut werden
- Nodes senden periodisch Benachrichtigungen über ihre Präsenz
- Nodes benachrichtigen die anderen Nodes, wenn sie offline gehen



<https://docs.ros.org/en/humble/Concepts.html>
<https://docs.ros.org/en/humble/Concepts/About-Quality-of-Service-Settings.html>

Das Robot Operating System ist ein Meta-Betriebssystem zur Entwicklung von Software für Roboter und zum Betrieb von Robotern.

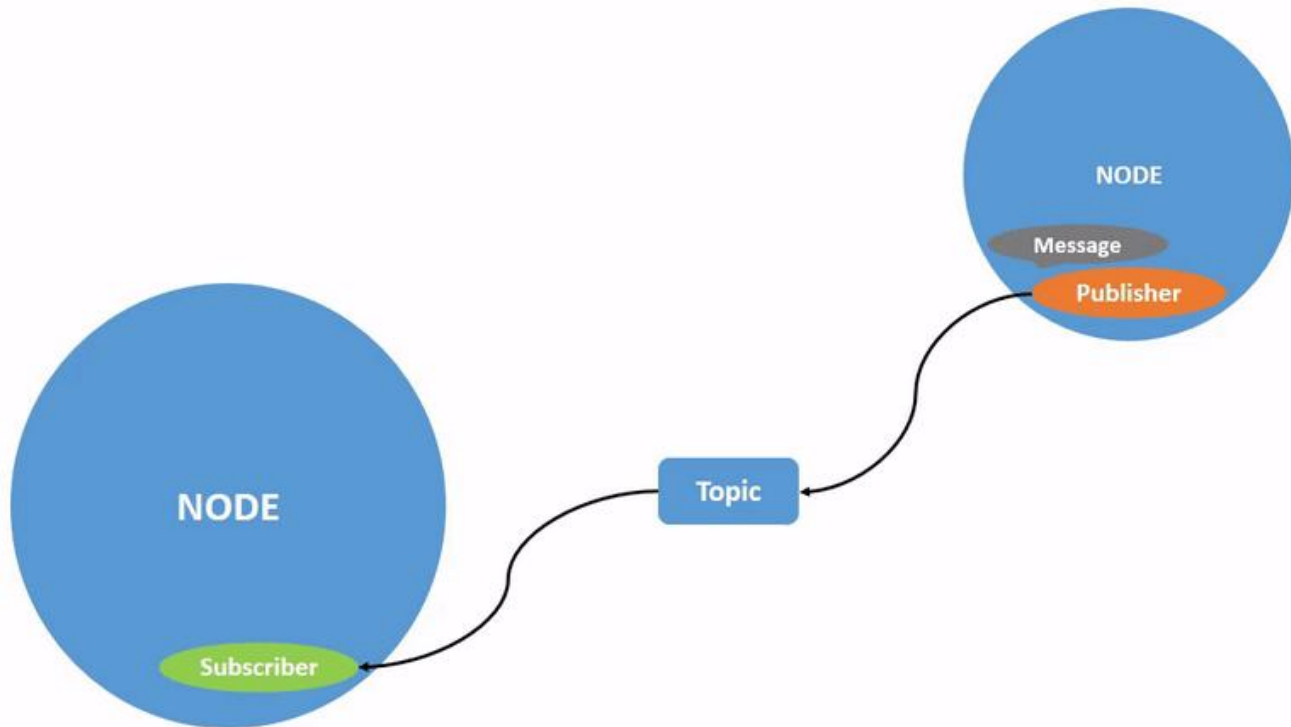
- Überblick über das ROS 2
- Packages
- Nodes
- **Kommunikation**
 - Topics
 - Messages
 - Services
- Parameter
- ROS Community
- Anwendungsbeispiele



<https://github.com/ros-infrastructure/artwork/blob/master/distributions/humble/HumbleHawksbill.png>

Mithilfe von Topics können Messages direkt zwischen unterschiedlichen Nodes versendet und empfangen werden.

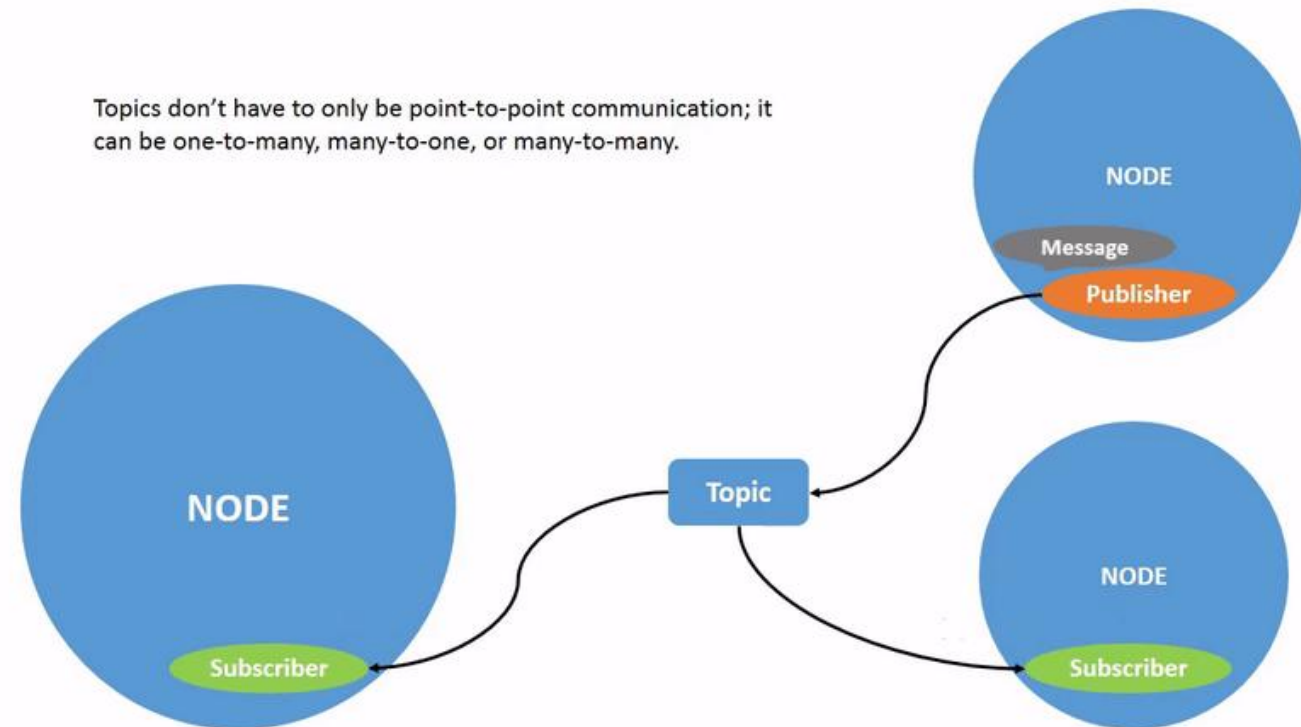
- Ein Topic ist ein Kanal zur Nachrichtenübertragung
- Die Nachrichtenübertragung erfolgt nach dem Publish-Subscribe Muster
- Peer-to-Peer: Direkte Übertragung von Messages (Nachrichten) zwischen den Nodes (mittels Publisher & Subscriber)
- Nur das Senden / Empfangen von Messages eines (zuvor) definierten Message Types ist möglich
- Publisher und Subscriber werden mithilfe einer ROS Client Library (z.B. rclpy) implementiert



<https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Topics/Understanding-ROS2-Topics.html>

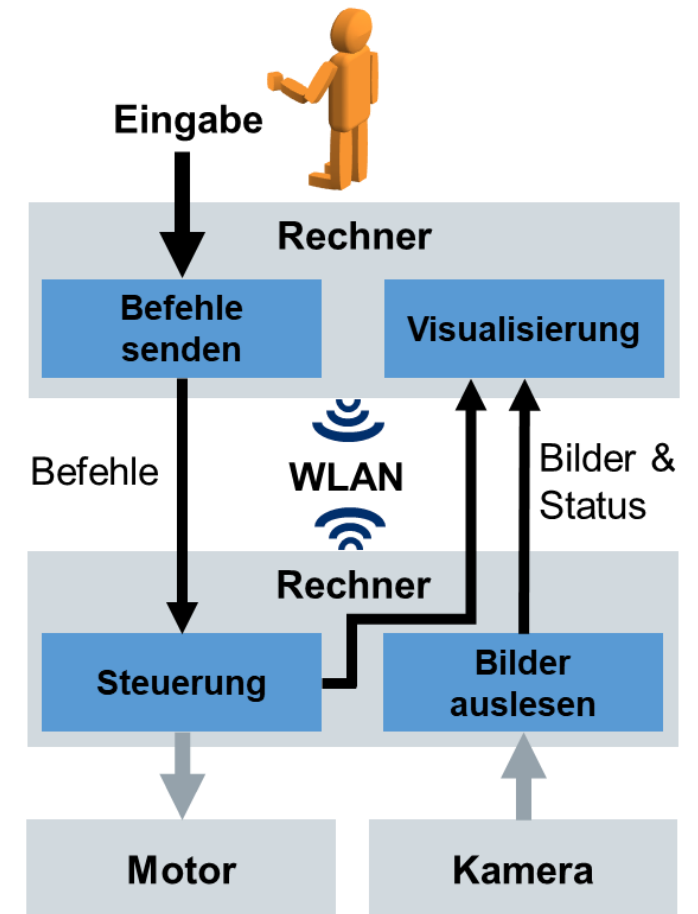
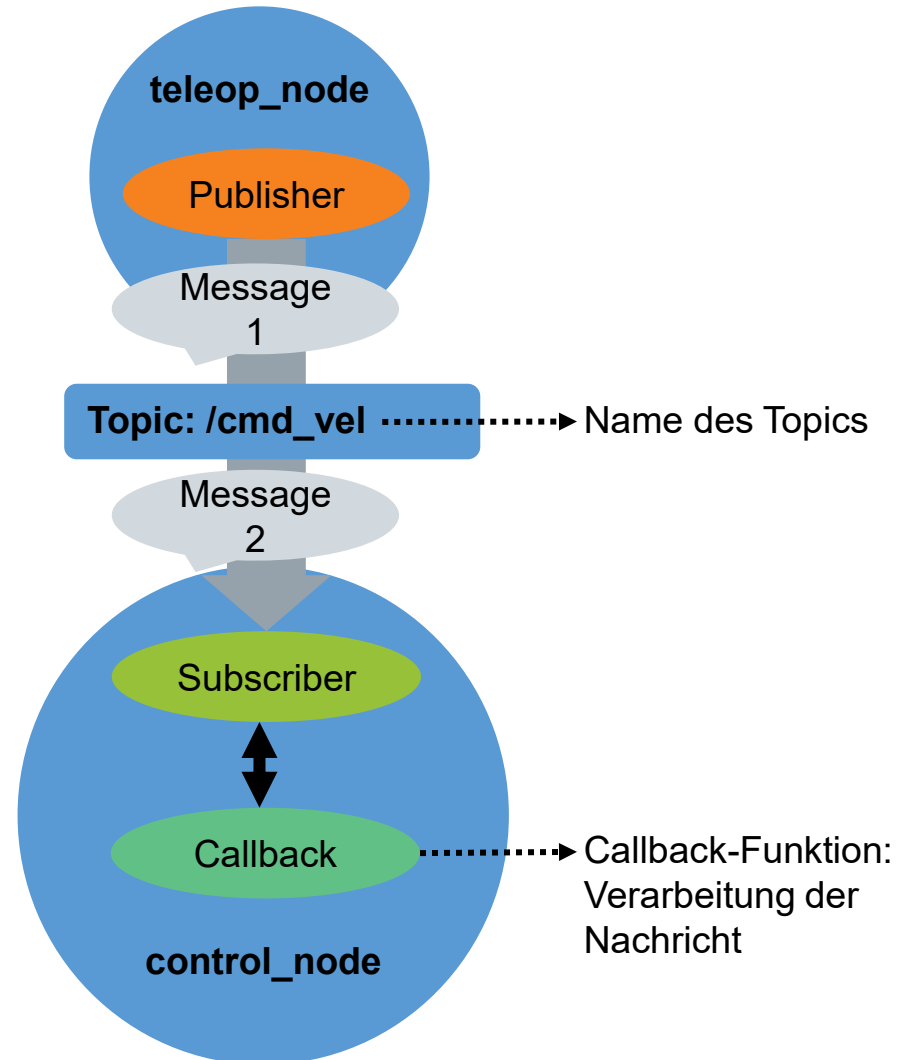
Mithilfe von Topics können Messages direkt zwischen unterschiedlichen Nodes versendet und empfangen werden.

- Ein Topic ist ein Kanal zur Nachrichtenübertragung
- Die Nachrichtenübertragung erfolgt nach dem Publish-Subscribe Muster
- Peer-to-Peer: Direkte Übertragung von Messages (Nachrichten) zwischen den Nodes (mittels Publisher & Subscriber)
- Nur das Senden / Empfangen von Messages eines (zuvor) definierten Message Types ist möglich
- Publisher und Subscriber werden mithilfe einer ROS Client Library (z.B. rclpy) implementiert
- Mehrere Subscriber können Messages eines Topics empfangen
- Mehrere Publisher können Messages über ein Topic versenden
- Gut geeignet für periodische Kommunikationen, z.B. Versenden von Sensordaten



<https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Topics/Understanding-ROS2-Topics.html>

Mithilfe von Publisher und Subscriber können Nodes über ein Topic Messages versenden und empfangen.



Das Robot Operating System ist ein Meta-Betriebssystem zur Entwicklung von Software für Roboter und zum Betrieb von Robotern.

- Überblick über das ROS 2
- Packages
- Nodes
- **Kommunikation**
 - Topics
 - **Messages**
 - Services
- Parameter
- ROS Community
- Anwendungsbeispiele



<https://github.com/ros-infrastructure/artwork/blob/master/distributions/humble/HumbleHawksbill.png>

Alle Kommunikationsarten des ROS basieren auf Messages.

- Messages werden mit der ROS interface definition language (IDL) in einer .msg-Datei definiert
- Messages sind unabhängig von der Programmiersprache
- Für jede Sprache wird basierend auf der Definition entsprechender Quellcode durch das ROS generiert
- Message Type:
Name des Packages / Name der .msg Datei
Datei: std_msgs/msg/String.msg → Typ: std_msgs/String
- Aufbau einer Message-Definition:

```

    fieldtype1 fieldname1
    fieldtype2 fieldname2
    fieldtype3 fieldname3

```

...

- Als fieldtype möglich sind
 - Ein built-in-type (auch als Array)
 - Andere Message Types, z.B. “geometry_msgs/TwistStamped“

Built-in-types currently supported:

Type name	C++	Python	DDS type
bool	bool	builtins.bool	boolean
byte	uint8_t	builtins.bytes*	octet
char	char	builtins.str*	char
float32	float	builtins.float*	float
float64	double	builtins.float*	double
int8	int8_t	builtins.int*	octet
uint8	uint8_t	builtins.int*	octet
int16	int16_t	builtins.int*	short
uint16	uint16_t	builtins.int*	unsigned short
int32	int32_t	builtins.int*	long
uint32	uint32_t	builtins.int*	unsigned long
int64	int64_t	builtins.int*	long long
uint64	uint64_t	builtins.int*	unsigned long long
string	std::string	builtins.str	string
wstring	std::u16string	builtins.str	wstring

<https://docs.ros.org/en/humble/Concepts/About-ROS-Interfaces.html>

Alle Kommunikationsarten des ROS basieren auf Messages.

- Messages werden mit der ROS interface definition language (IDL) in einer .msg-Datei definiert
- Messages sind unabhängig von der Programmiersprache
- Für jede Sprache wird basierend auf der Definition entsprechender Quellcode durch das ROS generiert
- Message Type:
 - Name des Packages / Name der .msg Datei
 - Datei: std_msgs/msg/String.msg → Typ: std_msgs/String
- Aufbau einer Message-Definition:
 - Header (optional):
 - stamp: Zeitstempel der Message
 - frame_id: Name des Koordinatensystems, in dem die Daten beschrieben sind
 - Datenstruktur

std_msgs/msg/Header Message

File: `std_msgs/msg/Header.msg`

Raw Message Definition

```
# Standard metadata for higher-level stamped data types.
# This is generally used to communicate timestamped data
# in a particular coordinate frame.

# Two-integer timestamp that is expressed as seconds and nanoseconds.
builtin_interfaces/Time stamp

# Transform frame with which this data is associated.

string frame_id
```

Compact Message Definition

```
builtin_interfaces/msg/Time stamp
string frame_id
```

https://github.com/ros2/common_interfaces/tree/humble
https://docs.ros2.org/latest/api/std_msgs/msg/Header.html

Message Types definieren die Art der Informationen, die versendet werden.

geometry_msgs/msg/TwistStamped

File: `geometry_msgs/msg/TwistStamped.msg`

Raw Message Definition

```
# A twist with reference coordinate frame and timestamp

std_msgs/Header header
Twist twist
```

Compact Message Definition

```
std_msgs/msg/Header header
geometry_msgs/msg/Twist twist
```

- Der TwistStamped Message Type beinhaltet einen Geschwindigkeitsbefehl: $[v_x, v_y, v_z, \omega_x, \omega_y, \omega_z]^T$

geometry_msgs/msg/Twist

File: `geometry_msgs/msg/Twist.msg`

Raw Message Definition

```
# This expresses velocity in free space broken into i

Vector3 linear
Vector3 angular
```

Compact Message Definition

```
geometry_msgs/msg/Vector3 linear
geometry_msgs/msg/Vector3 angular
```

geometry_msgs/msg/Vector3

File: `geometry_msgs/msg/Vector3.msg`

Raw Message Definition

```
# This represents a vector in free space.

# This is semantically different than a point.
# A vector is always anchored at the origin.
# When a transform is applied to a vector, only the rota

float64 x
float64 y
float64 z
```

Compact Message Definition

```
double x
double y
double z
```

https://docs.ros2.org/latest/api/geometry_msgs/msg/TwistStamped.html
https://docs.ros2.org/latest/api/geometry_msgs/msg/Twist.html
https://docs.ros2.org/latest/api/geometry_msgs/msg/Vector3.html

Das ROS stellt bereits eine Vielzahl an unterschiedlichen, robotikspezifischen Nachrichtentypen zur Verfügung.

Package sensor_msgs:

Messages (.msg)

- **BatteryState**: Describes the power state of the battery.
- **CameraInfo**: Meta information for a camera.
- **ChannelFloat32**: Holds optional data associated with each point in a PointCloud message.
- **CompressedImage**: A compressed image.
- **FluidPressure**: Single pressure reading for fluids (air, water, etc) like atmospheric and barometric pressure.
- **Illuminance**: Single photometric illuminance measurement.
- **Image**: An uncompressed image.
- **Imu**: Holds data from an IMU (Inertial Measurement Unit).
- **JointState**: Holds data to describe the state of a set of torque controlled joints.
- **JoyFeedbackArray**: An array of JoyFeedback messages.
- **JoyFeedback**: Describes user feedback in a joystick, like an LED, rumble pad, or buzzer.
- **Joy**: Reports the state of a joystick's axes and buttons.
- **LaserEcho**: A submessage of MultiEchoLaserScan and is not intended to be used separately.
- **LaserScan**: Single scan from a planar laser range-finder.
- **MagneticField**: Measurement of the Magnetic Field vector at a specific location.
- **MultiDOFJointState**: Representation of state for joints with multiple degrees of freedom, following JointTrajectory.
- **MultiEchoLaserScan**: Single scan from a multi-echo planar laser range-finder.
- **NavSatFix**: Navigation Satellite fix for any Global Navigation Satellite System.
- **NavSatStatus**: Navigation Satellite fix status for any Global Navigation Satellite System.

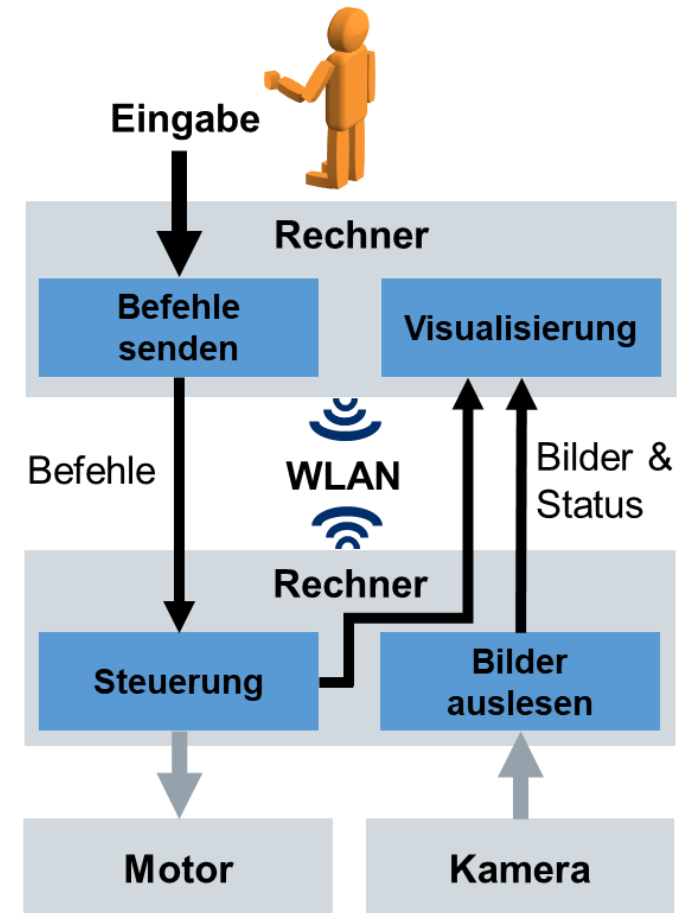
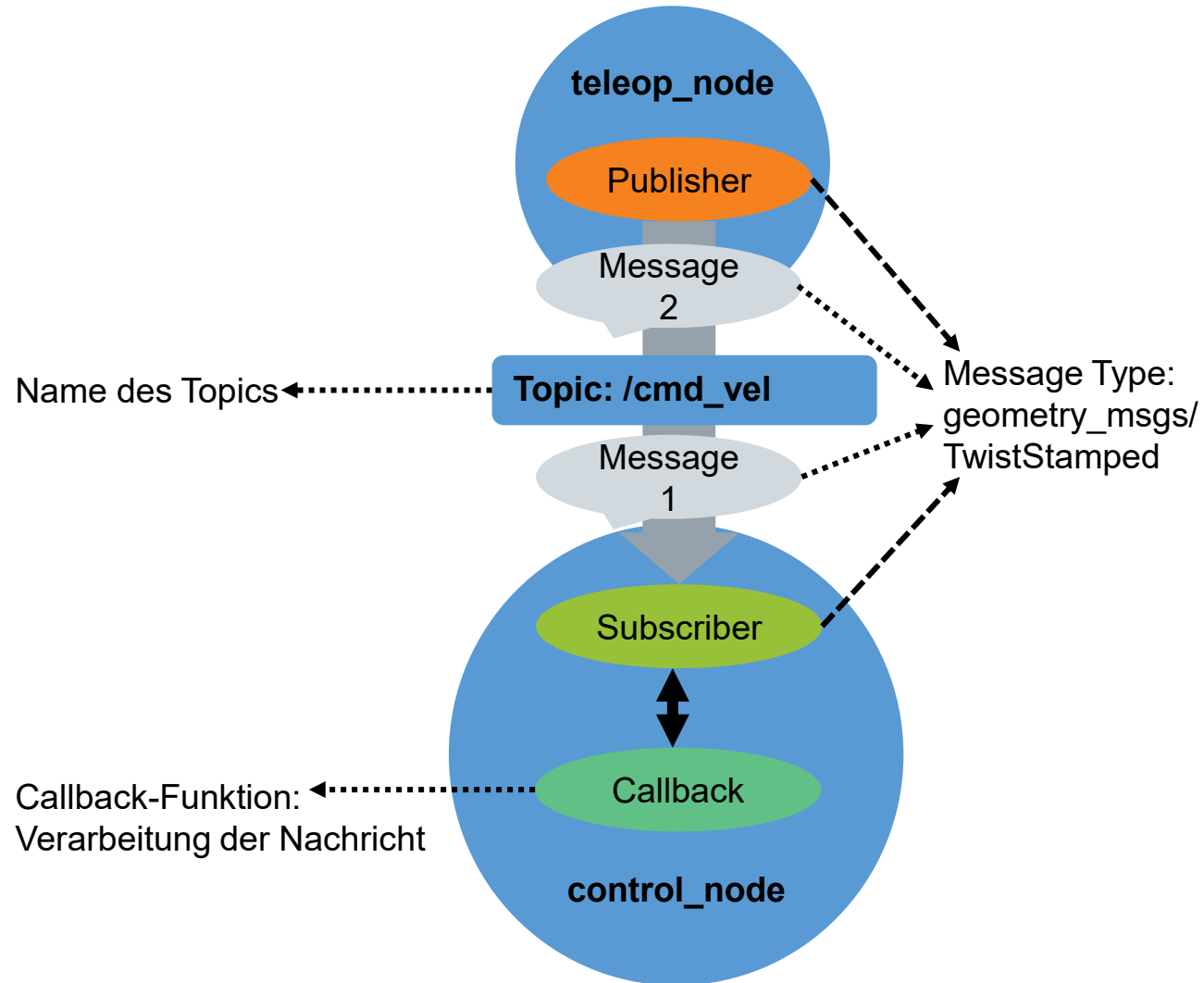
Package geometry_msgs:

Messages (.msg)

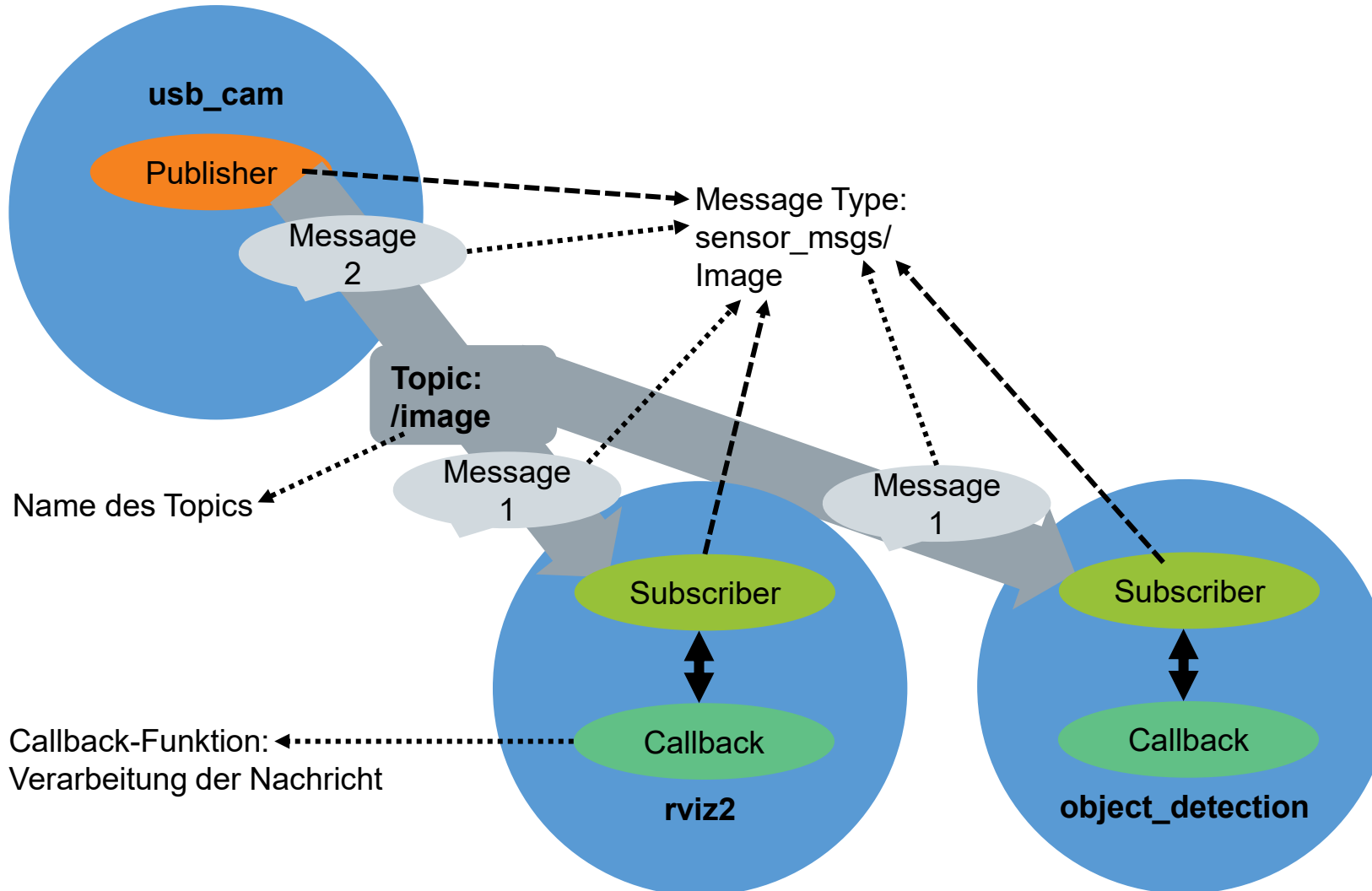
- **Accel**: Expresses acceleration in free space broken into its linear and angular parts.
- **AccelStamped**: An accel with reference coordinate frame and timestamp.
- **AccelWithCovariance**: Acceleration in free space with uncertainty.
- **AccelWithCovarianceStamped**: An estimated accel with reference coordinate frame and timestamp.
- **Inertia**: Expresses the inertial properties of a link.
- **InertiaStamped**: An Inertia with reference coordinate frame and timestamp.
- **Point32**: The position of a 3-dimensional point in free space, with 32-bit fields.
- **Point**: The position of a 3-dimensional point in free space.
- **PointStamped**: Point with reference coordinate frame and timestamp.
- **Polygon**: A specification of a polygon where the first and last points are assumed to be connected.
- **PolygonStamped**: A Polygon with reference coordinate frame and timestamp.
- **Pose2D**: **Deprecated as of Foxy and will potentially be removed in any following release.**
- **PoseArray**: An array of poses with a header for global reference.
- **Pose**: A representation of pose in free space, composed of position and orientation.
- **PoseStamped**: A Pose with reference coordinate frame and timestamp.
- **PoseWithCovariance**: A pose in free space with uncertainty.
- **PoseWithCovarianceStamped**: An estimated pose with a reference coordinate frame and timestamp.
- **Quaternion**: An orientation in free space in quaternion form.
- **QuaternionStamped**: An orientation with reference coordinate frame and timestamp.

https://github.com/ros2/common_interfaces/tree/humble/sensor_msgs
https://github.com/ros2/common_interfaces/tree/humble/geometry_msgs

Der Message Type von Messages, welche über ein Topic versandt werden, muss festgelegt werden.



Der Message Type von Messages, welche über ein Topic versandt werden, muss festgelegt werden.



Das Robot Operating System ist ein Meta-Betriebssystem zur Entwicklung von Software für Roboter und zum Betrieb von Robotern.

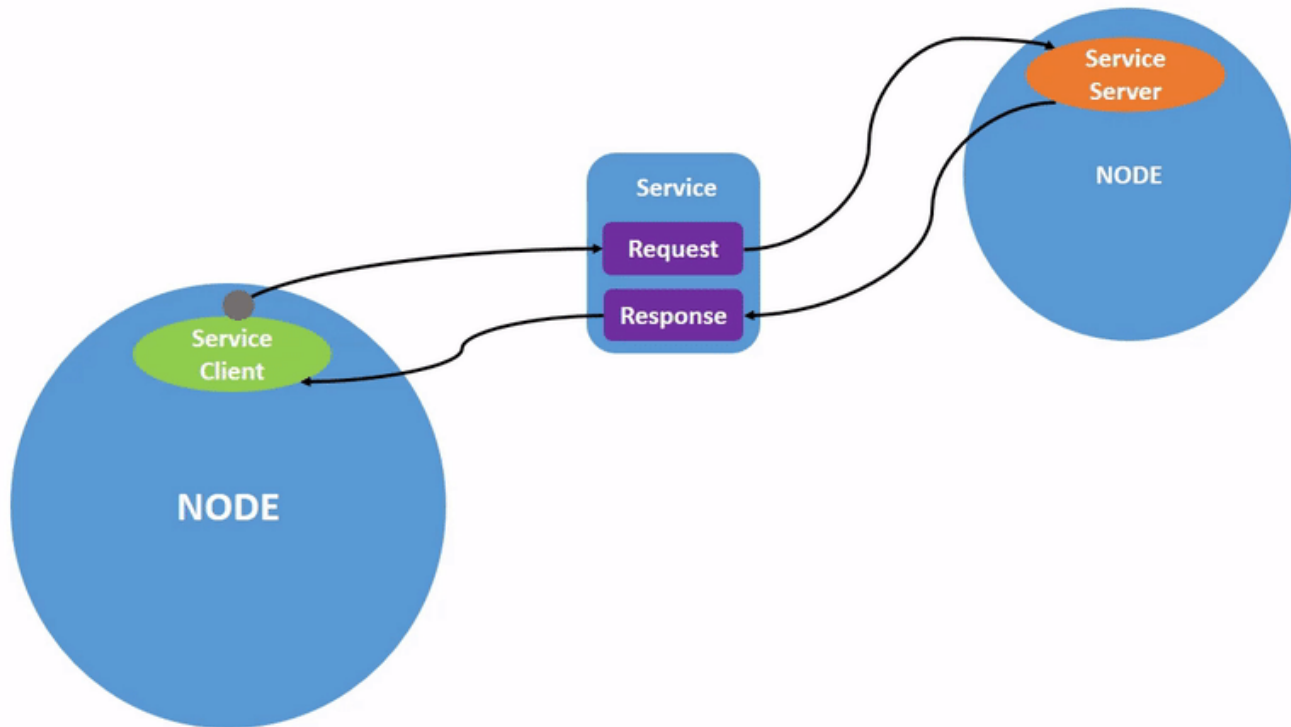
- Überblick über das ROS 2
- Packages
- Nodes
- **Kommunikation**
 - Topics
 - Messages
 - **Services**
- Parameter
- ROS Community
- Anwendungsbeispiele



<https://github.com/ros-infrastructure/artwork/blob/master/distributions/humble/HumbleHawksbill.png>

Ein ROS Service wird für den aperiodischen Austausch von Nachrichten eingesetzt.

- Ein Service wird von einem Node angeboten (Service Server)
- Die Kommunikation basiert auf dem Request/Response-Prinzip
- Ein Node kann eine Request-Message senden (Service Client)
- Auf den Request folgt die Abarbeitung des Services und die entsprechende Antwort (Response)
- Services werden durch ein Message-Paar in einer .srv Datei definiert (Service Type)



```
string str ← Request
---
string str ← Response
Message
```

- Gut geeignet für aperiodische Kommunikationen, kurze Aktionen

<https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Services/Understanding-ROS2-Services.html>

Ein Service ist durch eine Request Message und eine Response Message definiert.

`std_srvs/SetBool` Service

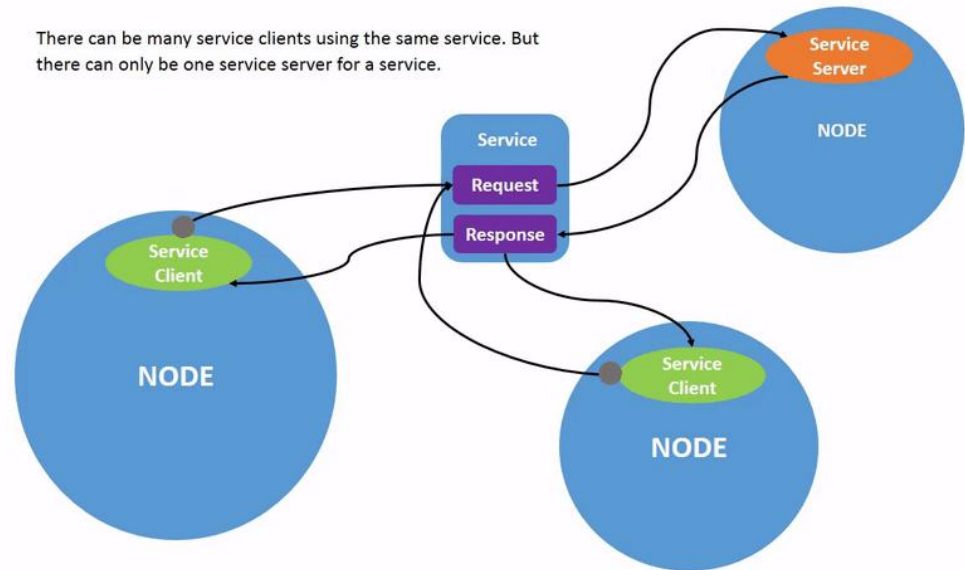
File: `std_srvs/SetBool.srv`

Raw Message Definition

```
bool data # e.g. for hardware enabling / disabling
---
bool success # indicate successful run of triggered service
string message # informational, e.g. for error messages
```

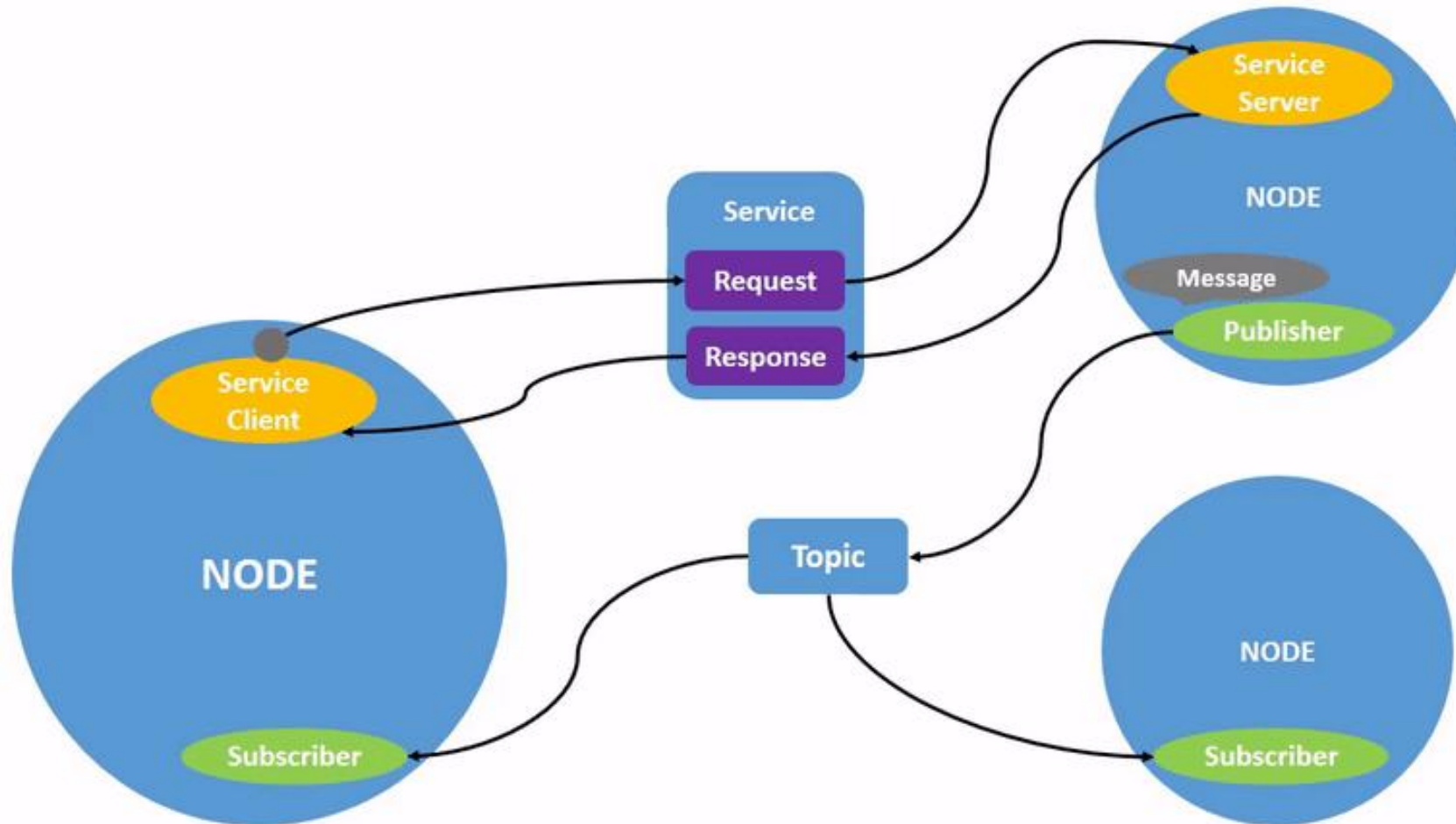
Compact Message Definition

<code>bool data</code>	← Request Message
<code>bool success</code> <code>string message</code>	← ResponseMessage



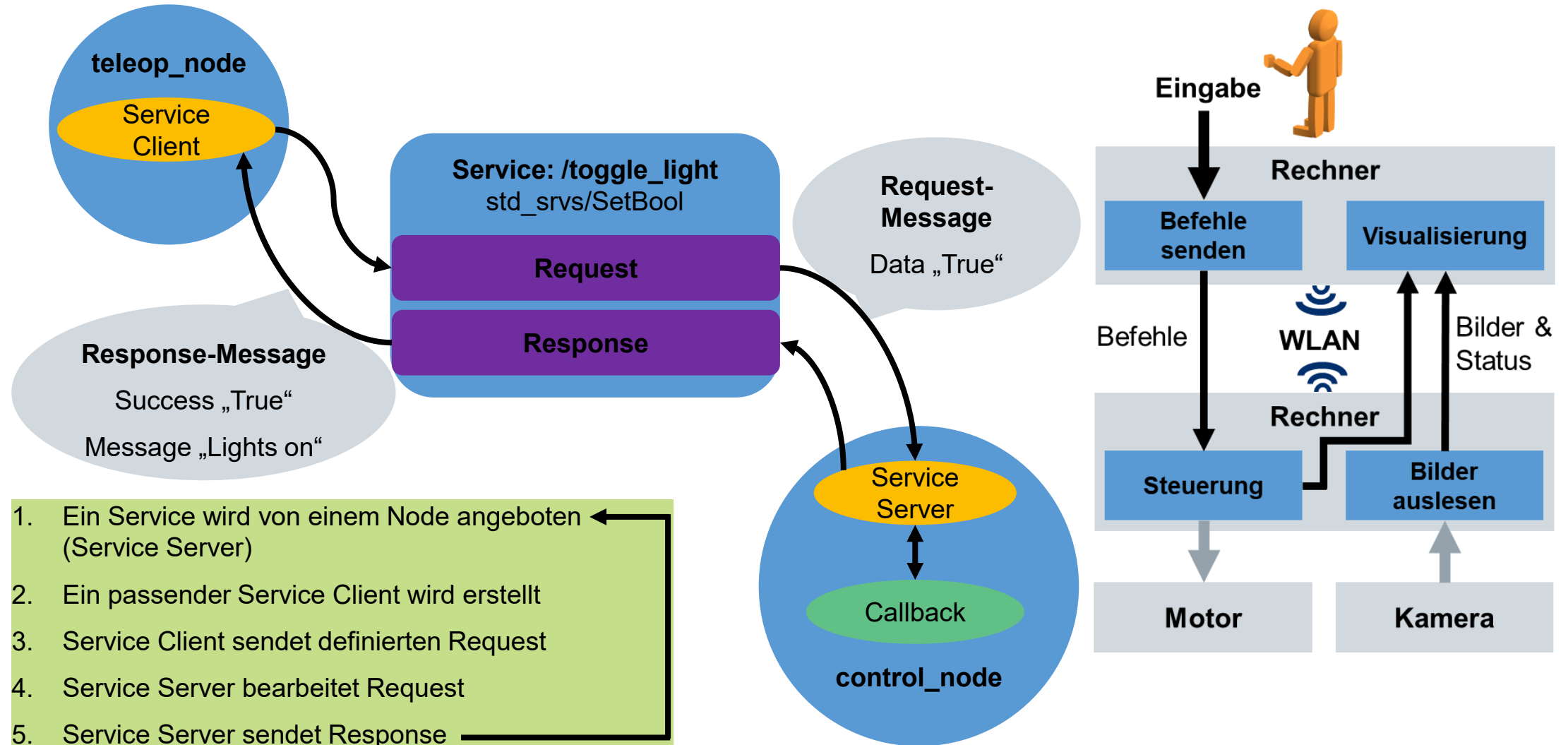
<https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Services/Understanding-ROS2-Services.html>

Services, Publisher und Subscriber können in einem Node kombiniert werden.



<http://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Nodes/Understanding-ROS2-Nodes.html>

Mithilfe eines Services kann eine bidirektionale Kommunikation aufgebaut werden.



Das Robot Operating System ist ein Meta-Betriebssystem zur Entwicklung von Software für Roboter und zum Betrieb von Robotern.

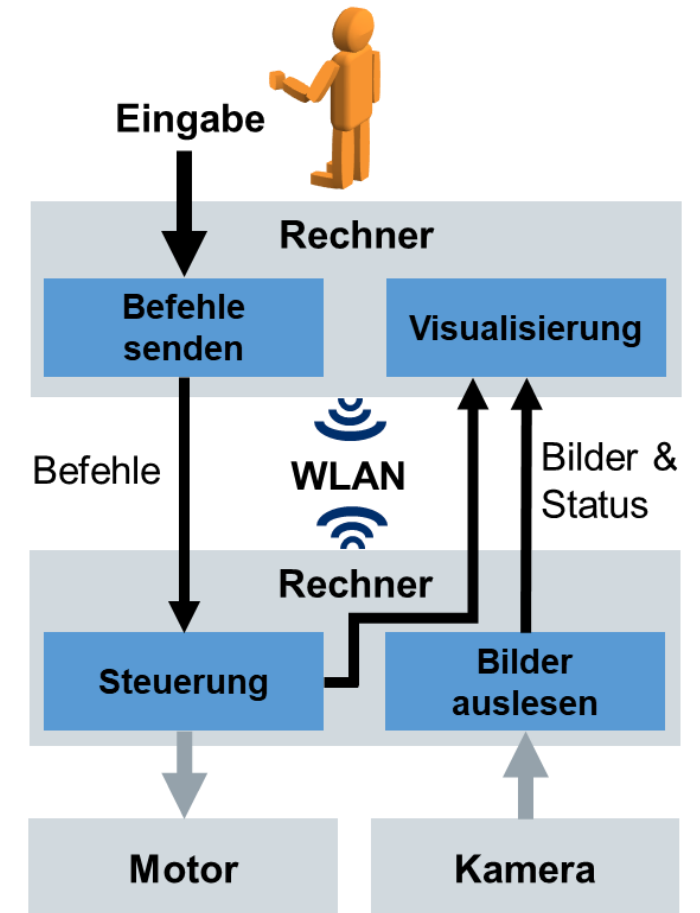
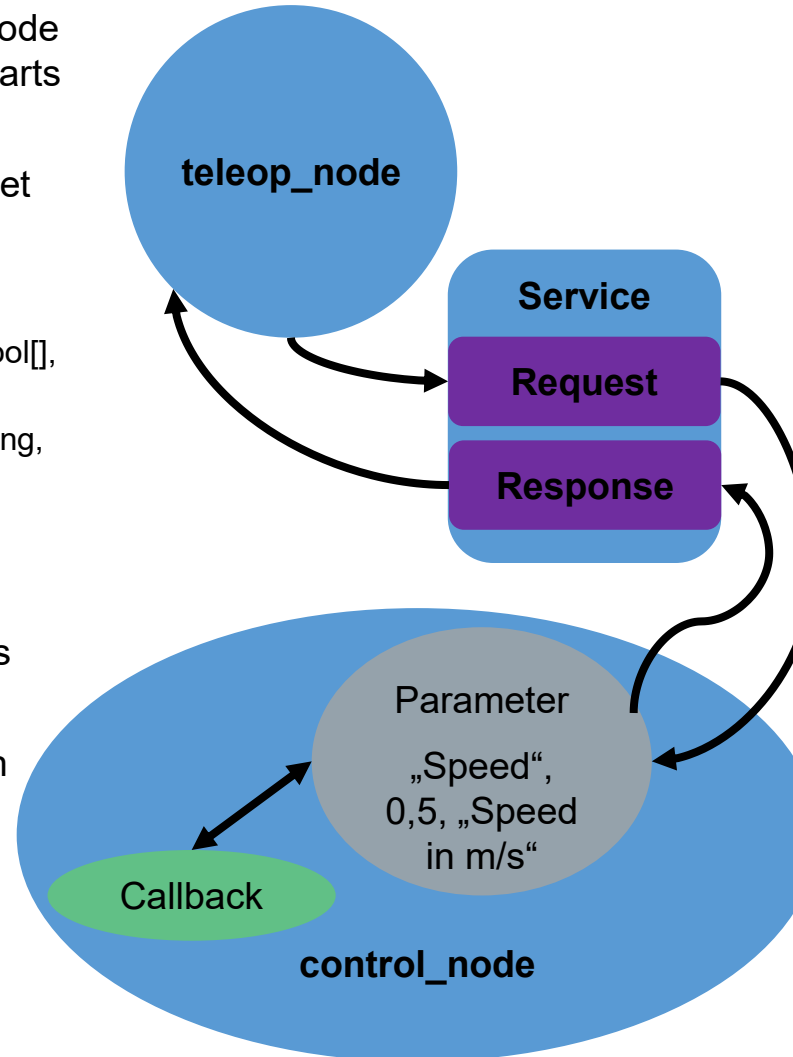
- Überblick über das ROS 2
- Packages
- Nodes
- Kommunikation
 - Topics
 - Messages
 - Services
- **Parameter**
- ROS Community
- Anwendungsbeispiele



<https://github.com/ros-infrastructure/artwork/blob/master/distributions/humble/HumbleHawksbill.png>

Parameter speichern grundlegende Werte und Einstellungen eines Nodes.

- ROS Parameter dienen dazu, einen Node und dessen Verhalten während des Starts und der Laufzeit zu konfigurieren
- Parameter sind einem Node zugeordnet
- Parameter bestehen aus
 - Key (String)
 - Value (bool, int64, float64, string, byte[], bool[], int64[], float64[] or string[])
 - Descriptor (optional, Parameterbeschreibung, z.B. maximaler Wert)
- Parameter müssen explizit deklariert werden (rcldpy, rclcpp)
- Parameter können von anderen Nodes gelesen und geändert werden (rcl)
- Parameter können Callbacks auslösen



<https://docs.ros.org/en/humble/Concepts/About-ROS-2-Parameters.html>

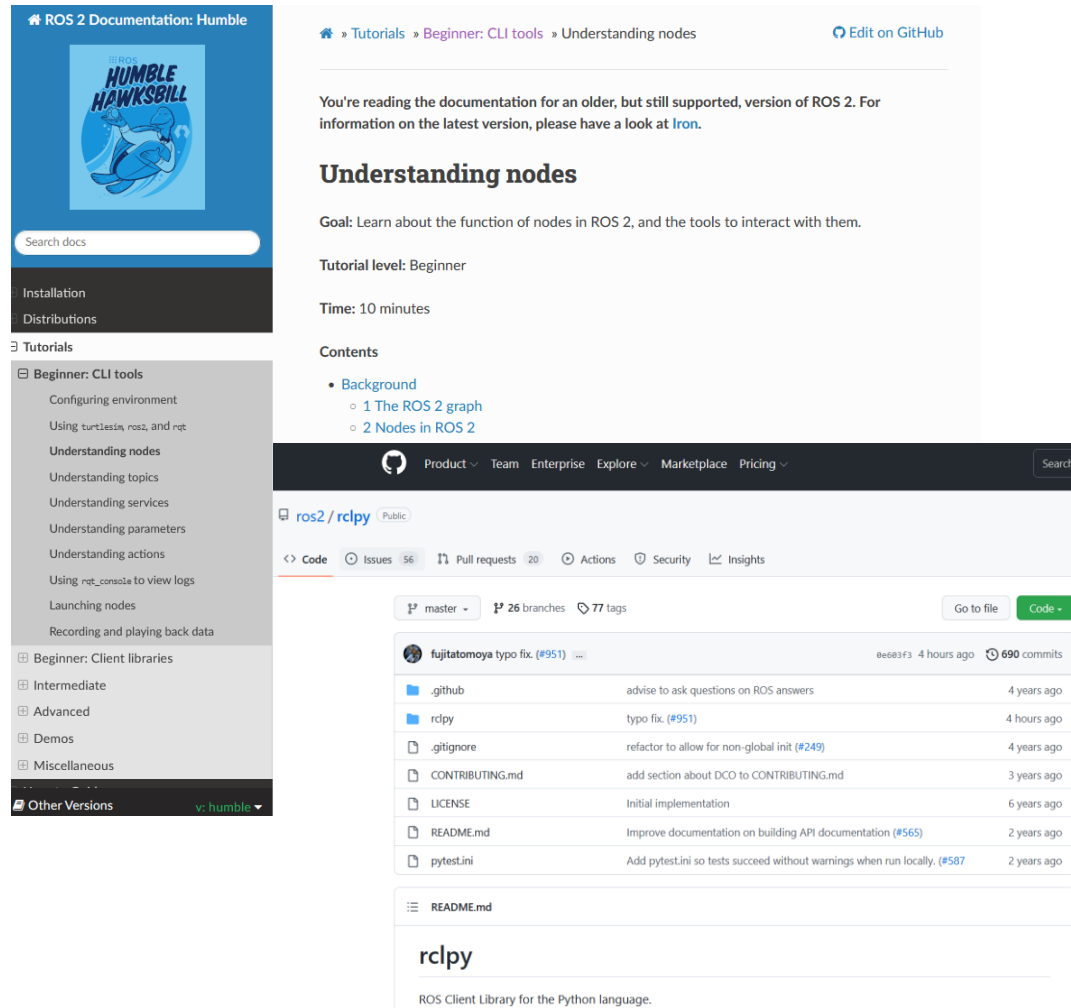
Das Robot Operating System ist ein Meta-Betriebssystem zur Entwicklung von Software für Roboter und zum Betrieb von Robotern.

- Überblick über das ROS 2
- Packages
- Nodes
- Kommunikation
 - Topics
 - Messages
 - Services
- Parameter
- **ROS Community**
- Anwendungsbeispiele



<https://github.com/ros-infrastructure/artwork/blob/master/distributions/humble/HumbleHawksbill.png>

Im Internet sind viele Informationen, Tutorials und Hilfestellungen zum ROS verfügbar.



ROS 2 Documentation: Humble Hawksbill

Search docs

Installation
Distributions

Tutorials

- Beginner: CLI tools
 - Configuring environment
 - Using turtlesim, roslaunch, and rqt
 - Understanding nodes
 - Understanding topics
 - Understanding services
 - Understanding parameters
 - Understanding actions
 - Using rqt_console to view logs
 - Launching nodes
 - Recording and playing back data
- Intermediate
- Advanced
- Demos
- Miscellaneous

Other Versions: v: humble

Understanding nodes

You're reading the documentation for an older, but still supported, version of ROS 2. For information on the latest version, please have a look at [Iron](#).

Goal: Learn about the function of nodes in ROS 2, and the tools to interact with them.

Tutorial level: Beginner

Time: 10 minutes

Contents

- Background
 - 1 The ROS 2 graph
 - 2 Nodes in ROS 2

ros2/rclpy (Public)

Code Issues 56 Pull requests 20 Actions Security Insights

master 26 branches 77 tags

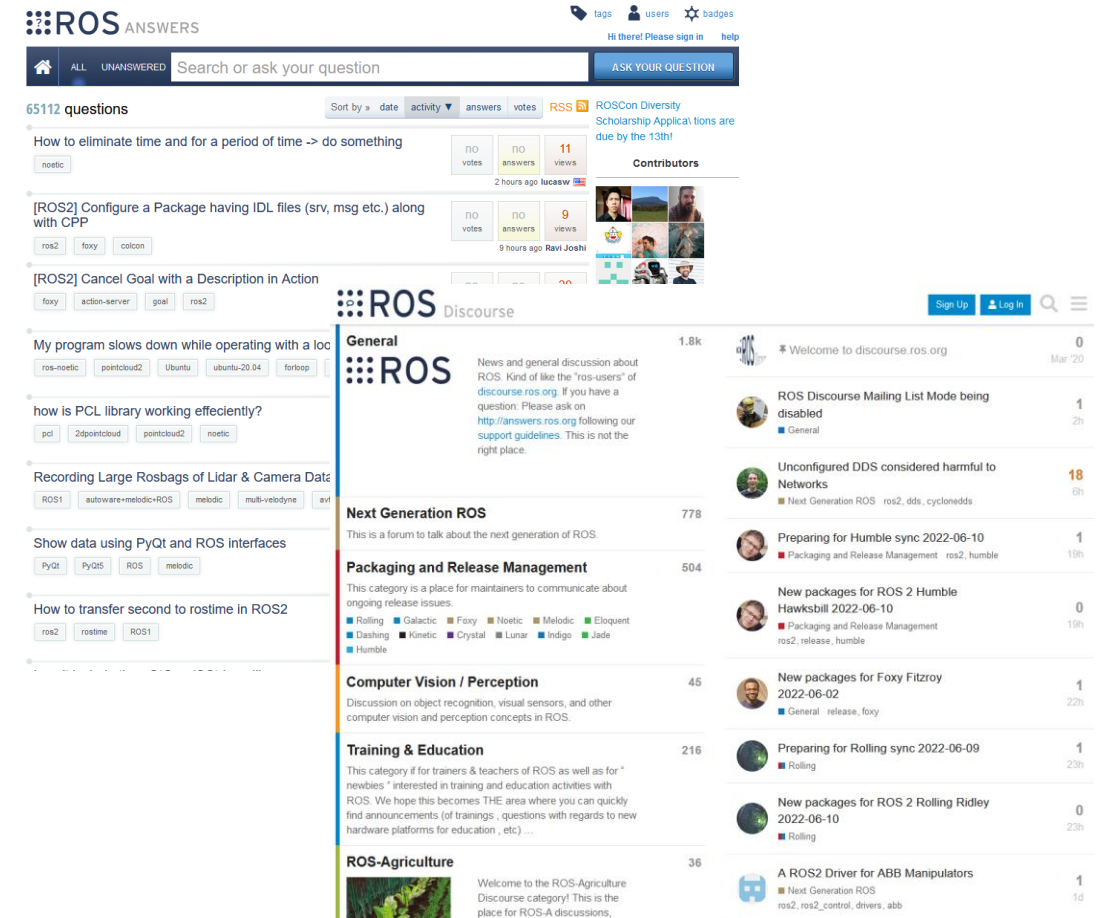
fujitatomoya typo fix (#951) 4 hours ago 690 commits

File	Description	Time
.github	advise to ask questions on ROS answers	4 years ago
rclpy	typo fix (#951)	4 hours ago
.gitignore	refactor to allow for non-global init (#249)	4 years ago
CONTRIBUTING.md	add section about DCO to CONTRIBUTING.md	3 years ago
LICENSE	Initial implementation	6 years ago
README.md	Improve documentation on building API documentation (#565)	2 years ago
pytest.ini	Add pytest.ini so tests succeed without warnings when run locally. (#587)	2 years ago

README.md

rclpy

ROS Client Library for the Python language.



ROS ANSWERS

Search or ask your question

65112 questions

Sort by: date activity answers votes RSS

How to eliminate time and for a period of time -> do something

no votes no answers 11 views

2 hours ago fucaww

[ROS2] Configure a Package having IDL files (srv, msg etc.) along with CPP

no votes no answers 9 views

9 hours ago Ravi Joshi

[ROS2] Cancel Goal with a Description in Action

foxy action-server goal ros2

My program slows down while operating with a local

ros-noetic pointcloud2 Ubuntu ubuntu-20.04 forloop

how is PCL library working efficiently?

pcl 2dpointcloud pointcloud2 noetic

Recording Large Rosbags of Lidar & Camera Data

ROS1 autoware-melodic+ROS melodic multi-velodyne avi

Show data using PyQt and ROS interfaces

PyQt PyQIS ROS melodic

How to transfer second to rostime in ROS2

ros2 rostime ROS1

ROS Discourse

Sign Up Log In

General

News and general discussion about ROS. Kind of like the "ros-users" of [discourse.ros.org](http://answers.ros.org). If you have a question: Please ask on <http://answers.ros.org> following our support guidelines. This is not the right place.

Next Generation ROS

This is a forum to talk about the next generation of ROS.

Packaging and Release Management

This category is a place for maintainers to communicate about ongoing release issues.

Computer Vision / Perception

Discussion on object recognition, visual sensors, and other computer vision and perception concepts in ROS.

Training & Education

This category is for trainers & teachers of ROS as well as for "newbies" interested in training and education activities with ROS. We hope this becomes THE area where you can quickly find announcements (of trainings), questions with regards to new hardware platforms for education, etc. ...

ROS-Agriculture

Welcome to the ROS-Agriculture Discourse category! This is the place for ROS-A discussions.

ROS Discourse Mailing List Mode being disabled

General 1 2h

Unconfigured DDS considered harmful to Networks

Next Generation ROS ros2, dds, cyclonedds 18 1h

Preparing for Humble sync 2022-06-10

Packaging and Release Management ros2, humble 1 10h

New packages for ROS 2 Humble Hawksbill 2022-06-10

Packaging and Release Management ros2, release, humble 0 10h

New packages for Foxy Fitzroy 2022-06-02

General release, foxy 1 22h

Preparing for Rolling sync 2022-06-09

Rolling 1 23h

New packages for ROS 2 Rolling Ridley 2022-06-10

Rolling 0 23h

A ROS2 Driver for ABB Manipulators

Next Generation ROS ros2, ros2_control, drivers, abb 1 1d

<https://answers.ros.org/questions/>

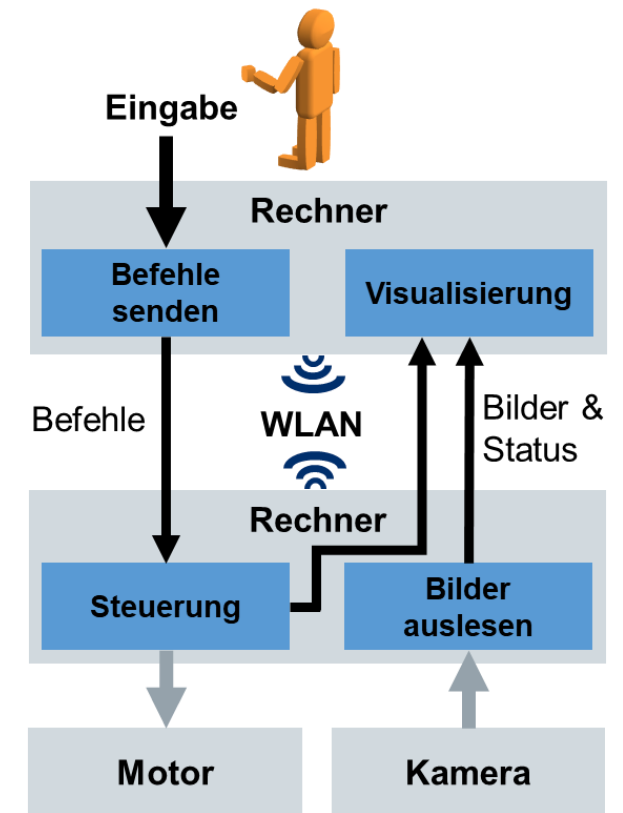
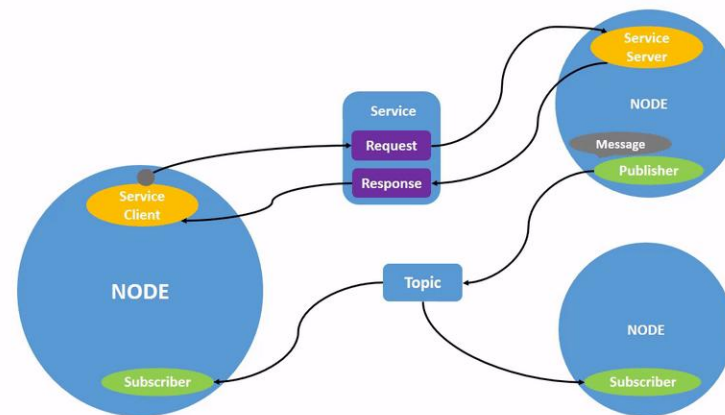
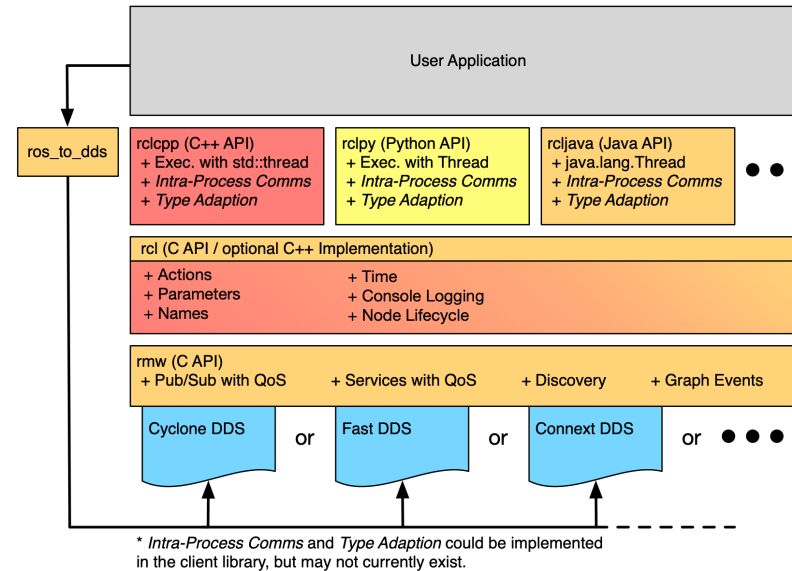
<https://discourse.ros.org/>

<https://github.com/ros2/rclpy>

<https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Nodes/Understanding-ROS2-Nodes.html>

Nach dieser Vorlesungseinheit sollten folgende Fragen beantwortet werden können.

- Was ist das ROS?
- Warum wird Software in Packages aufgeteilt?
- Was ist ein ROS Node?
- Wie funktioniert die Kommunikation über Topics?
- Wie ist eine ROS Message aufgebaut?
- Wie funktioniert die Kommunikation mit ROS Services?
- Was sind Parameter im ROS?



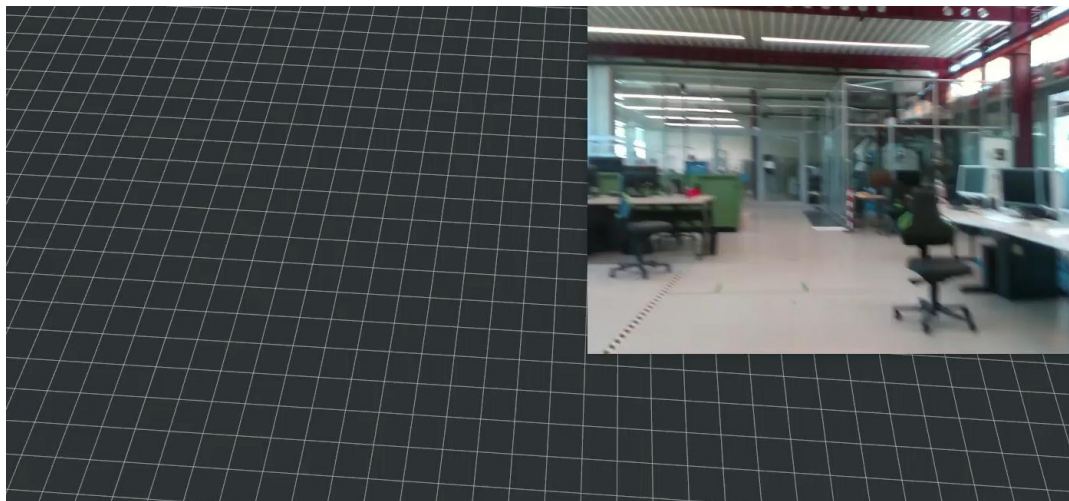
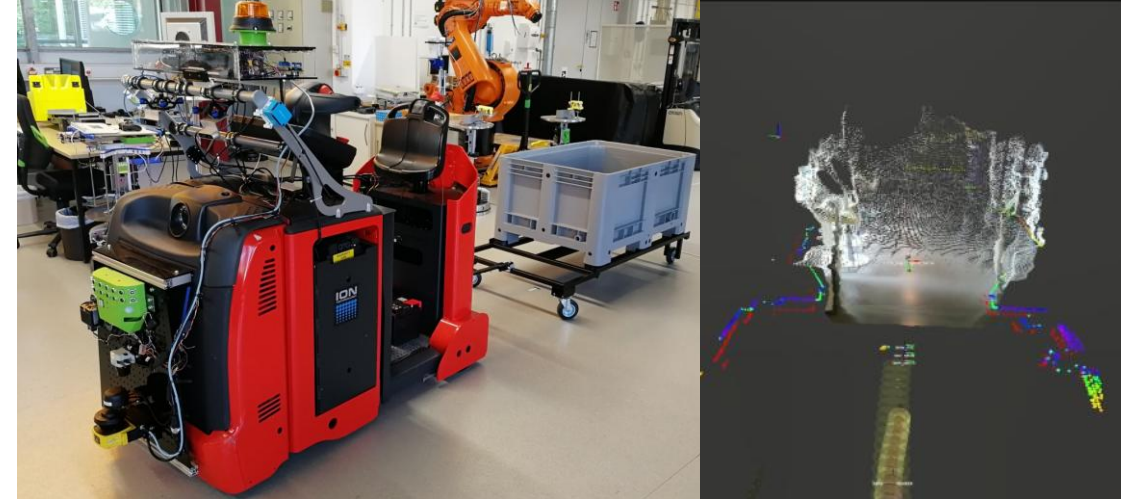
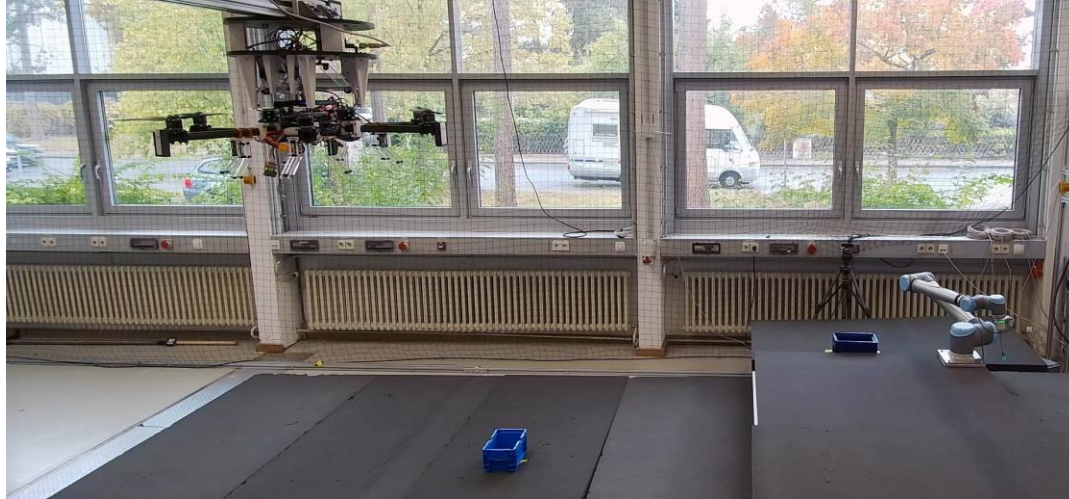
Das Robot Operating System ist ein Meta-Betriebssystem zur Entwicklung von Software für Roboter und zum Betrieb von Robotern.

- Überblick über das ROS 2
- Packages
- Nodes
- Kommunikation
 - Topics
 - Messages
 - Services
- Parameter
- ROS Community
- Anwendungsbeispiele



<https://github.com/ros-infrastructure/artwork/blob/master/distributions/humble/HumbleHawksbill.png>

Das ROS ist als Framework für die Programmierung und den Betrieb von Robotern vielfältig einsetzbar.

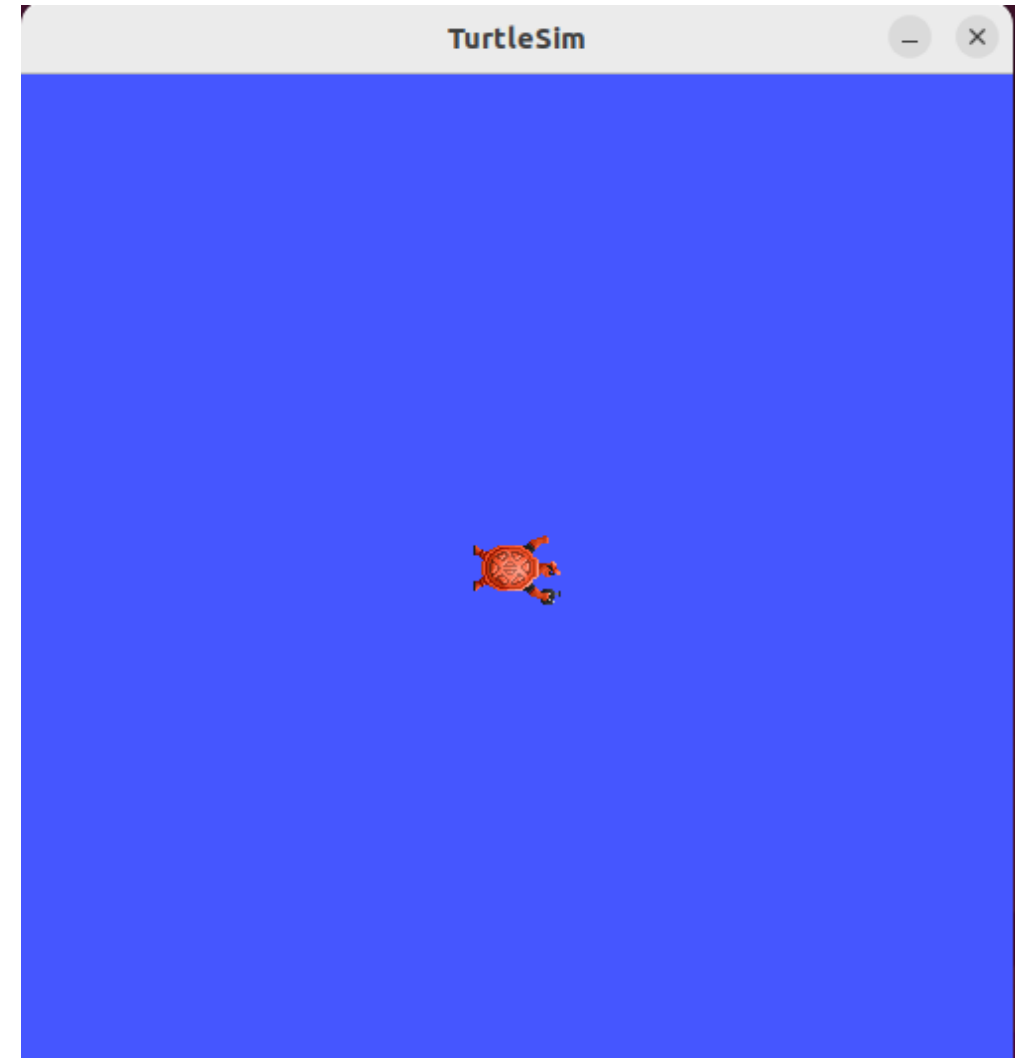


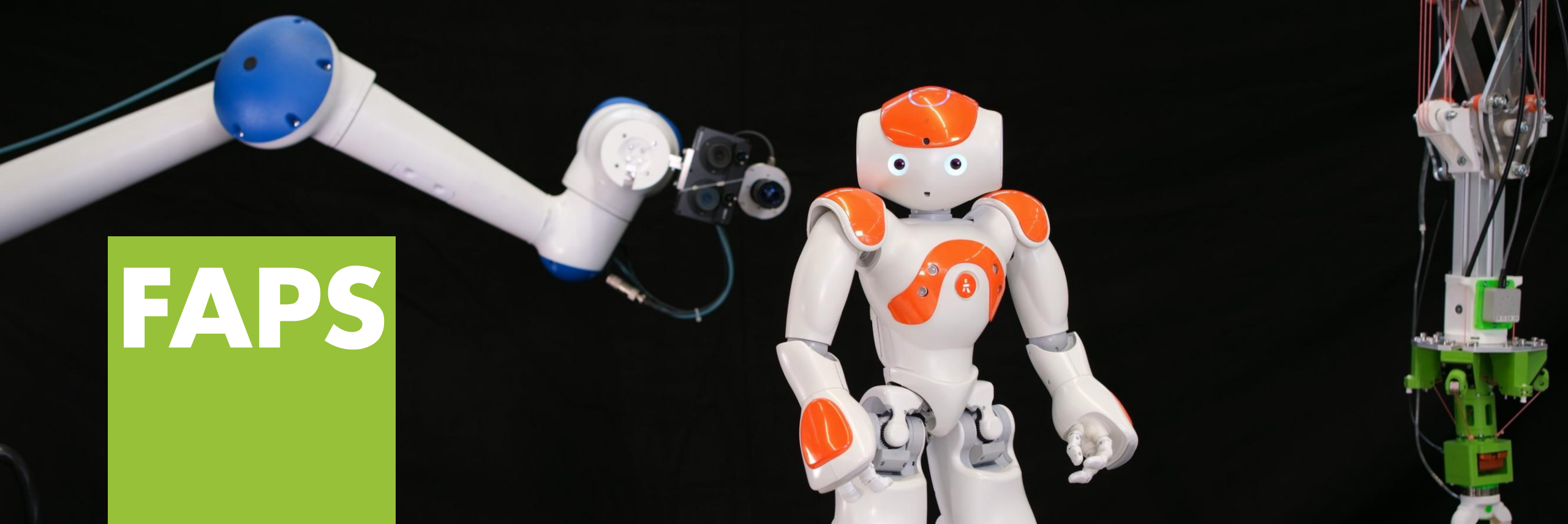
In der Übung zu dieser Vorlesungseinheit wird der praktische Einsatz des ROS vorgestellt.

```

1 import rclpy
2 from rclpy.node import Node
3 from std_msgs.msg import String
4
5 class GdrPublisher(Node):
6
7     def __init__(self):
8         super().__init__('gdr_publisher')
9         self.publisher_ = self.create_publisher(String, 'topic', 10)
10        timer_period = 0.5 # seconds
11        self.timer = self.create_timer(timer_period, self.timer_callback)
12        self.i = 0
13
14    def timer_callback(self):
15        msg = String()
16        msg.data = 'Hello World: %d' % self.i
17        self.publisher_.publish(msg)
18        self.get_logger().info('Publishing: "%s"' % msg.data)
19        self.i += 1
20
21 def main(args=None):
22     rclpy.init(args=args)
23
24     gdr_publisher = GdrPublisher()
25
26     rclpy.spin(gdr_publisher)
27
28     # Destroy the node explicitly
29     # (optional - otherwise it will be done automatically
30     # when the garbage collector destroys the node object)
31     gdr_publisher.destroy_node()
32     rclpy.shutdown()
33
34 if __name__ == '__main__':
35     main()

```





Prof. Dr.-Ing. Jörg Franke

**Lehrstuhl für Fertigungsautomatisierung
und Produktionssystematik**

Friedrich-Alexander-Universität Erlangen-Nürnberg



Friedrich-Alexander-Universität
Technische Fakultät

DANKE